

# Agenti basati su conoscenza

*Come costruire agenti dotati di  
capacità inferenziali*

Claudio Gallicchio, 2025/2026  
*[claudio.gallicchio@unipi.it](mailto:claudio.gallicchio@unipi.it)*

# Introduzione alla parte 2 del corso

Abbiamo visto:

- agenti con stato e con obiettivo in mondi osservabili con stati atomici e azioni descrivibili in maniera semplice
- enfasi sul processo di ricerca

# Di cosa parleremo...

- Come migliorare le capacità di ragionamento degli agenti, dotandoli di rappresentazioni di mondi complessi e astratti, non descrivibili semplicemente
- Agenti *basati su conoscenza*, dotati di una KB (*Knowledge Base*) con conoscenza espressa in maniera esplicita e dichiarativa

# Riferimenti

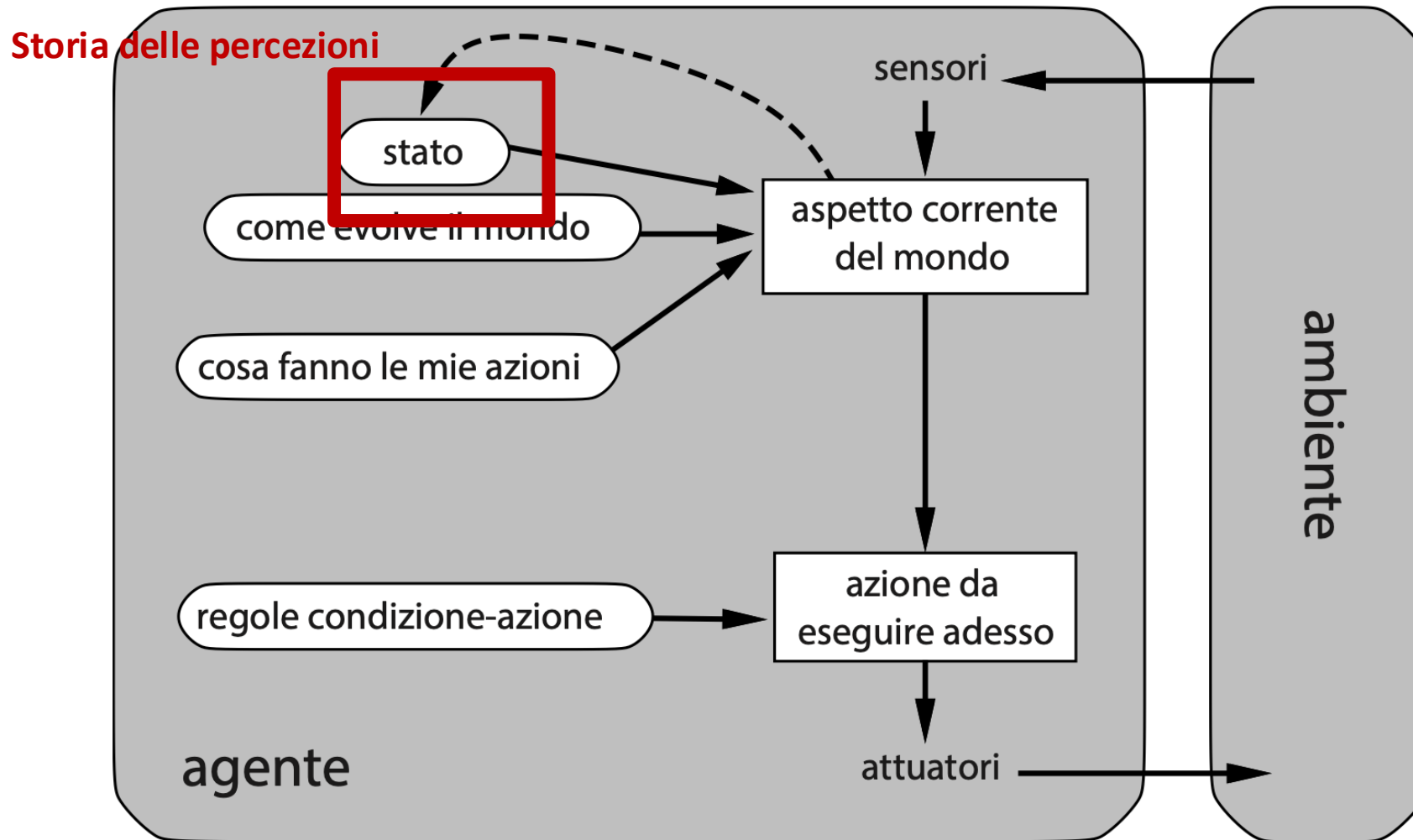


AIMA Cap 7

■ 7.1 – 7.3

# Introduzione agli agenti basati su conoscenza

# Agenti basati su modello



L'agente mantiene uno stato interno che serve per tenere traccia della parte del mondo che non può vedere all'istante corrente.

Lo stato dipende dalla storia delle percezioni.

# Agenti “Knowledge Based”

- I problemi di IA richiedono una conoscenza intensiva.
- Rappresentiamo il mondo in modo **astratto** per funzionalità specifiche degli agenti.
- Per ambienti complessi, necessitiamo di **linguaggi per rappresentare la conoscenza che siano espressivi e di capacità di inferenza**.
- La conoscenza può essere inserita manualmente, acquisita da testi, esperienze o esperti.

# Approccio dichiarativo

- La KB racchiude tutta la conoscenza necessaria a decidere l'azione da compiere in forma *dichiarativa*
- Un agente basato su conoscenza può essere costruito dicendogli (TELL) ciò che deve sapere.
  - Si inizia con una base di conoscenza vuota
  - Si aggiungono progressivamente formule alla base di conoscenza, una alla volta

# Approccio dichiarativo vs procedurale

- La KB racchiude tutta la conoscenza necessaria a decidere l'azione da compiere in forma *dichiarativa*
- L'alternativa (*approccio procedurale*) è scrivere un programma che implementa il processo decisionale, una volta per tutte.
- Un agente KB è più flessibile: più semplice acquisire conoscenza incrementalmente e modificare il comportamento con l'esperienza

A large, furry Wumpus with glowing eyes and sharp teeth stands on the left side of the image. The background is a dark, atmospheric scene of ancient stone ruins and a complex labyrinth of stone walls and steps. The lighting is dim, with some warm light coming from arched doorways in the distance. Several small, dark birds are flying in the sky above the Wumpus.

# Il mondo del Wumpus

un esempio concreto

# Il mondo del Wumpus: un esempio



Il mondo del Wumpus è una caverna fatta di stanze connesse tra di loro da passaggi.

# Il mondo del Wumpus: un esempio



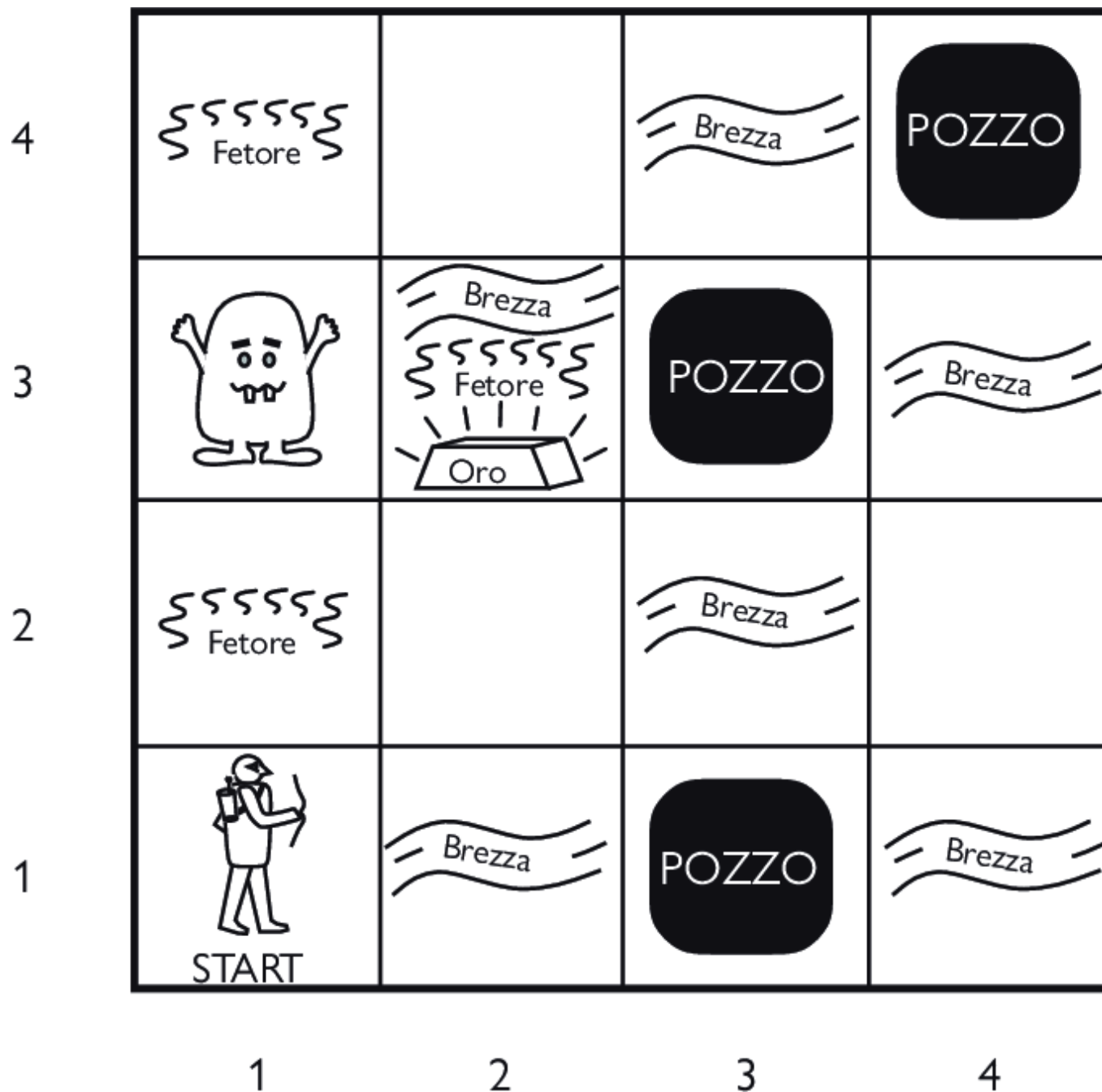
Il wumpus è una **bestia puzzolente** che mangia chiunque entri nella stanza in cui si trova.

Il **wumpus può essere ucciso dall'agente**, che ha *solo una freccia* a disposizione.

# Il mondo del Wumpus: un esempio



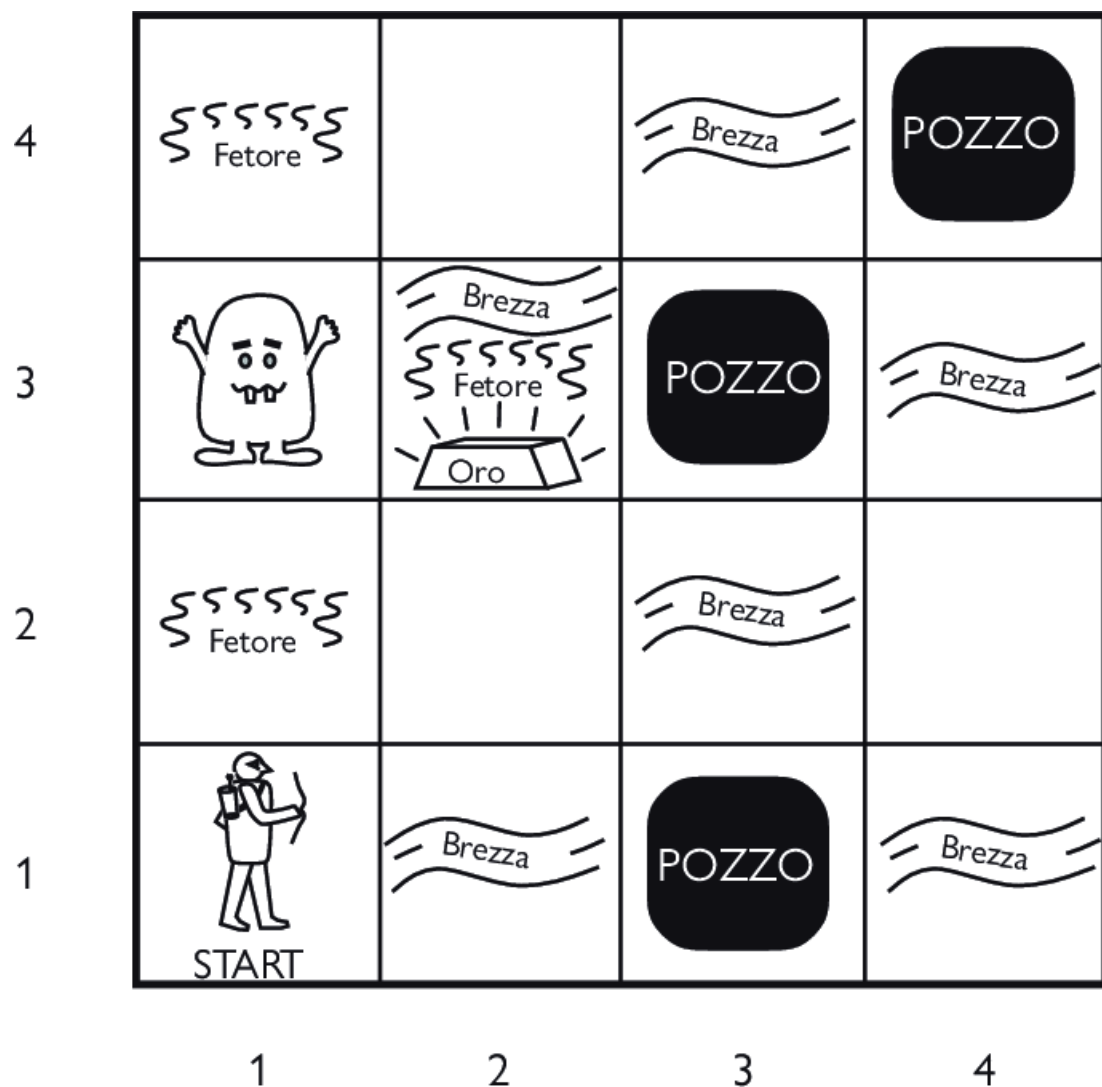
- Ci sono stanze con dei pozzi: se l'agente entra in una di queste stanze cade nel pozzo e muore.
  - Il Wumpus non muore nel pozzo.
- In una delle stanze si trova l'oro e l'obiettivo dell'agente è di trovare l'oro e tornare a casa con l'oro, sano e salvo.
- L'agente non conosce l'ambiente, né la sua locazione. Solo all'inizio sa dove si trova (in [1,1]).



# Il mondo del Wumpus: un esempio

Definizione precisa  
dell'ambiente fornita sotto  
forma di descrizione PEAS...

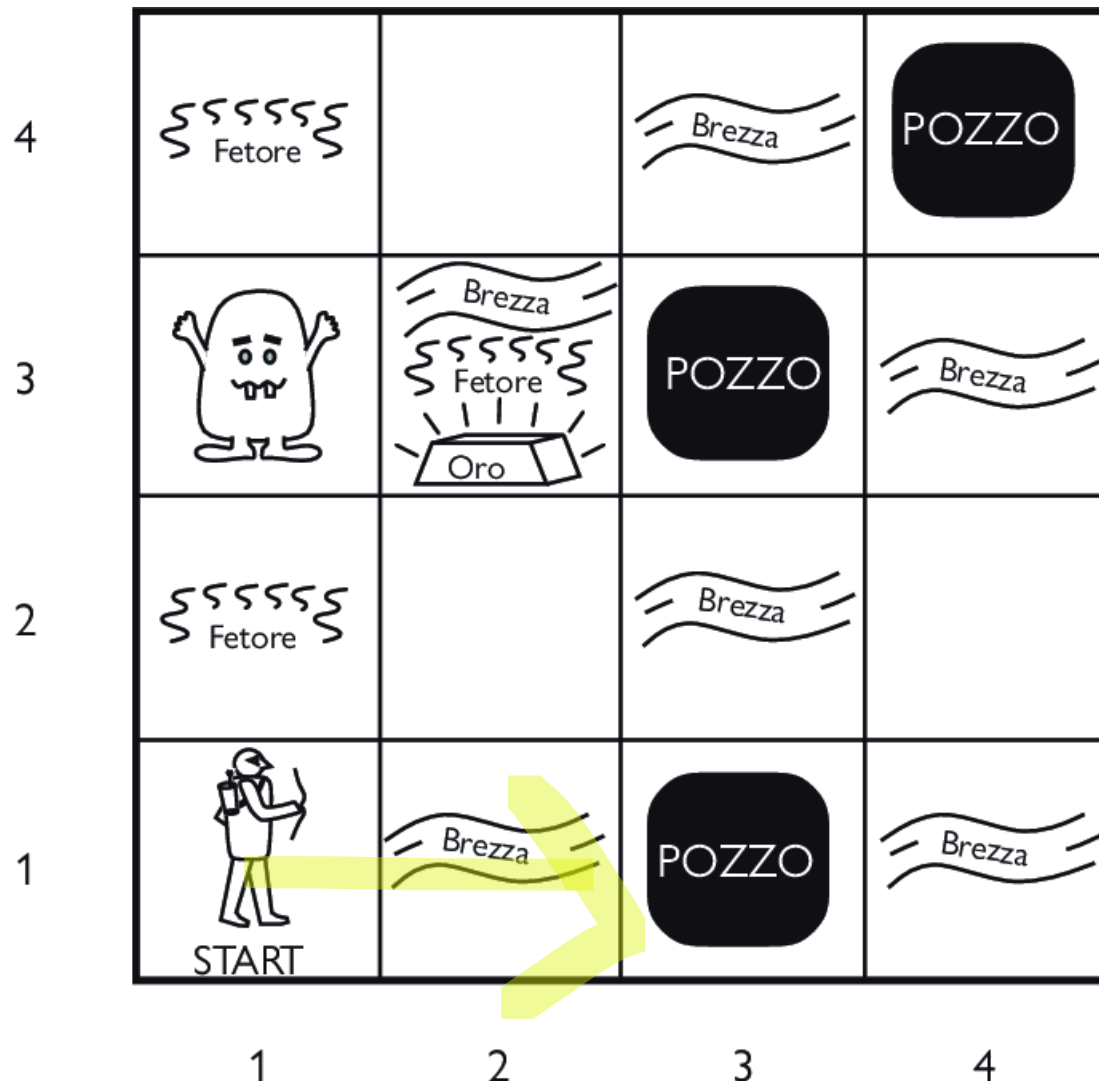
# Il mondo del Wumpus: un esempio



## Misura di prestazione:

- **+1000** se trova l'oro, torna in [1,1] e esce;
- **-1000** se muore;
- **-1** per ogni azione;
- **-10** se usa la freccia.

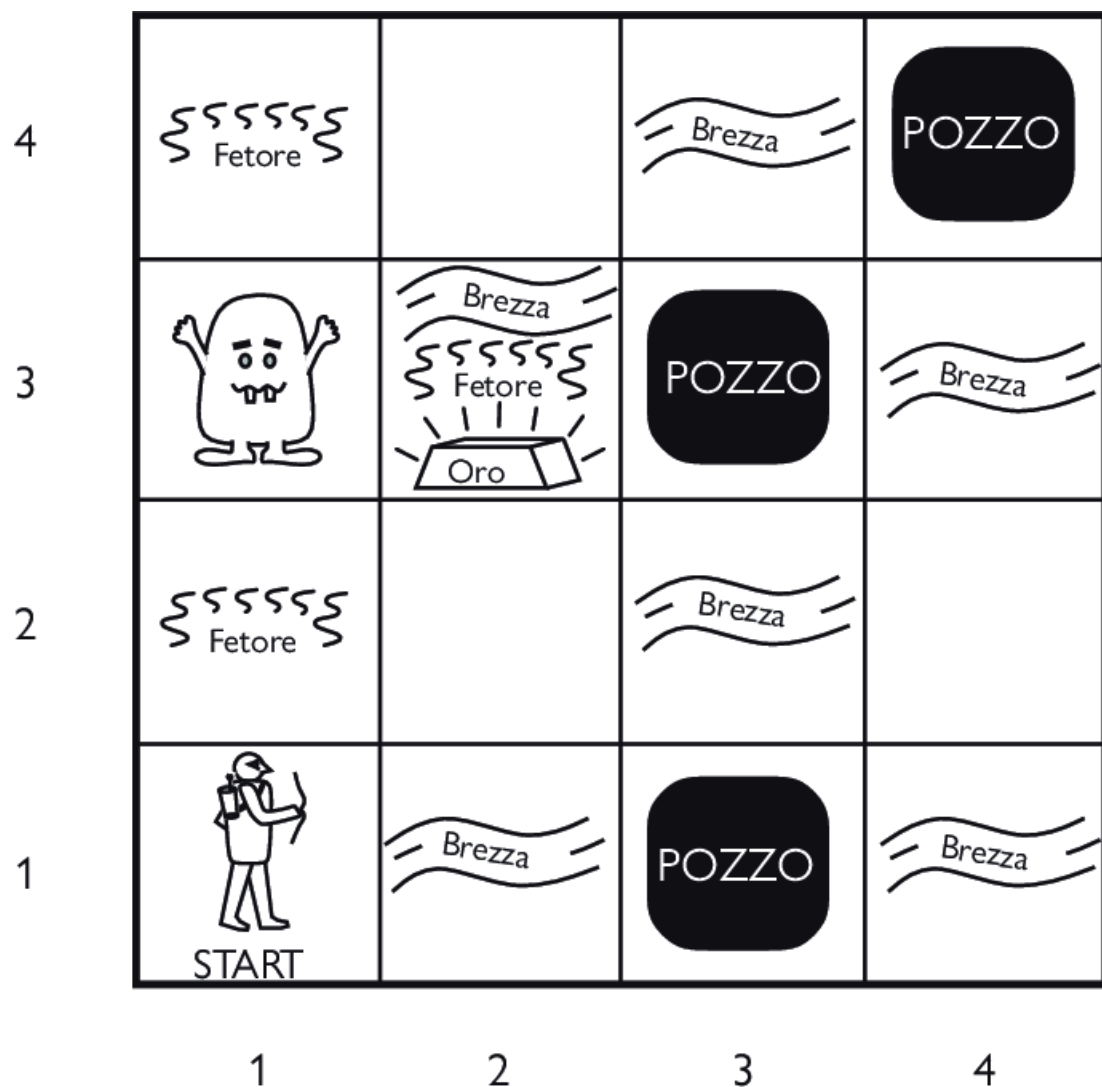
# Il mondo del Wumpus: un esempio



## Ambiente:

- Griglia 4 x 4 di stanze, circondata da pareti di delimitazione.
- L'agente comincia sempre dalla posizione [1,1], rivolto verso est (dx).
  - [1,1] e' safe
- Le posizioni dell'oro e del Wumpus sono scelte **casualmente** tra tutti i riquadri tranne quello iniziale.
- Tutti i riquadri (tranne quello iniziale) hanno una **probabilità 0.2** di contenere un pozzo.

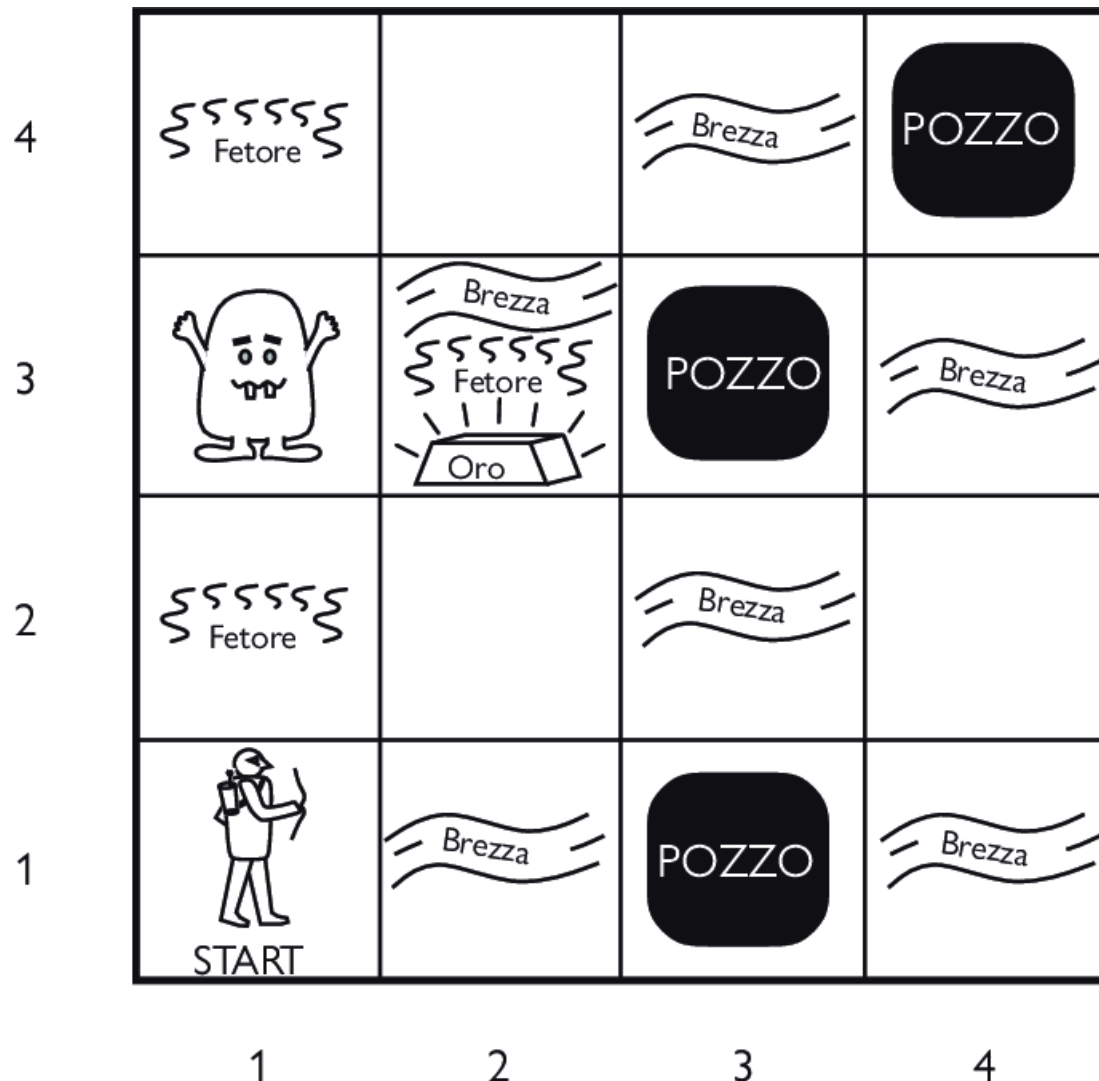
# Il mondo del Wumpus: un esempio



## Azioni (attuatori):

- *avanti*
- *a destra* di 90°, *a sinistra* di 90°
- *afferra* un oggetto
- *scaglia la freccia* (solo una)
- *esce*

# Il mondo del Wumpus: un esempio



## Percezioni (sensori):

- *fetore* nelle caselle adiacenti al Wumpus;
- *brezza* nelle caselle adiacenti ai pozzi;
- *luccichio* nelle caselle con l'oro;
- *bump* se sbatte in un muro;
- *urlo* se il Wumpus viene ucciso.
- L'agente *non percepisce* la sua locazione.

# Il mondo del Wumpus: uno scenario (1)

|                |           |     |     |
|----------------|-----------|-----|-----|
| 1,4            | 2,4       | 3,4 | 4,4 |
| 1,3            | 2,3       | 3,3 | 4,3 |
| 1,2<br>OK      | 2,2       | 3,2 | 4,2 |
| 1,1<br>A<br>OK | 2,1<br>OK | 3,1 | 4,1 |

(a)

- Le percezioni sono una quintupla [*Fetore*, *Brezza*, *Luccichio*, *Bump*, *Urlo*]
- Percezione in [1,1]  
[*none*, *none*, *none*, *none*, *none*]
- Né *Brezza* né *Fetore* in [1,1], quindi [1,2] e [2,1] sono sicure (**OK**)
- L'agente decide di spostarsi in [2,1] ...

## Il mondo del Wumpus: uno scenario (2)

|                |                                                |           |     |
|----------------|------------------------------------------------|-----------|-----|
| 1,4            | 2,4                                            | 3,4       | 4,4 |
| 1,3            | 2,3                                            | 3,3       | 4,3 |
| 1,2<br>OK      | 2,2<br>P?                                      | 3,2       | 4,2 |
| 1,1<br>V<br>OK | 2,1<br><div>A</div> <div>B</div> <div>OK</div> | 3,1<br>P? | 4,1 |

(b)

- Percezione in [2,1]  
[none, *Brezza*, none, none, none]
- L'agente percepisce una *Brezza*.  
Quindi c'è un *pozzo* in [2,2] o [3,1] (**P?**)
- L'agente **torna** in [1,1] e poi si sposta in [1,2] ...

## Il mondo del Wumpus: uno scenario (3)

|                     |                     |           |     |
|---------------------|---------------------|-----------|-----|
| 1,4                 | 2,4                 | 3,4       | 4,4 |
| 1,3<br>W!           | 2,3                 | 3,3       | 4,3 |
| 1,2<br>A<br>F<br>OK | 2,2<br>OK           | 3,2       | 4,2 |
| 1,1<br>V<br>OK      | 2,1<br>B<br>V<br>OK | 3,1<br>P! | 4,1 |

(a)

- Percezione in [1,2]  
[Fetore,none,none,none,none]
- Il Wumpus non può essere in [1,1] (*safe* per definizione), né in [2,2] (altrimenti avrebbe percepito *Fetore* anche in [2,1]). Quindi è in [1,3].
- Siccome non c'è *Brezza* in [1,2], [2,2] è OK e ci deve essere un *Pozzo* in [3,1].

## Il mondo del Wumpus: uno scenario (4)

|                     |                      |        |     |
|---------------------|----------------------|--------|-----|
| 1,4                 | 2,4<br>P?            | 3,4    | 4,4 |
| 1,3 W!              | 2,3<br>A<br>F L<br>B | 3,3 P? | 4,3 |
| 1,2<br>F<br>V<br>OK | 2,2<br>V<br>OK       | 3,2    | 4,2 |
| 1,1<br>V<br>OK      | 2,1<br>B<br>V<br>OK  | 3,1 P! | 4,1 |

(b)

- L'agente si sposta in [2,2] e poi in [2,3]
- Percezione in [2,3]  
[*Fetore*, *Brezza*, *Luccichio*, *none*, *none*]
- Percepisce un *Luccichio*, afferra l'Oro e torna sui suoi passi, percorrendo caselle OK.

Agenti basati sulla conoscenza

# Agente basato su conoscenza

- Un agente basato su conoscenza mantiene una *base di conoscenza* (KB): un insieme di *enunciati* (*formule*) espressi in un linguaggio di rappresentazione
- Interagisce con la KB mediante una interfaccia funzionale *Tell-Ask*:
  - *Tell*: per aggiungere nuovi enunciati a KB
  - *Ask*: per interrogare la KB
  - *Retract*: per eliminare enunciati

# Agente basato su conoscenza

- Gli enunciati nella KB rappresentano le opinioni/credenze dell'agente
- Le risposte  $\alpha$  devono essere tali che  $\alpha$  discende necessariamente dalla KB



# Il problema fondamentale da risolvere

*Il problema:* data una base di conoscenza KB, contenente una rappresentazione dei fatti che si **ritengono veri**, come dedurre che un certo fatto  $\alpha$  è vero di conseguenza?

$$KB \models \alpha \quad (\textit{conseguenza logica})$$

# Programma di un agente KB

**function** Agente-KB (*percezione*) **returns** *un'azione*

**persistent:** *KB*, una base di conoscenza

*t*, un contatore, inizialmente a 0, che indica il tempo

TELL(*KB*, Costruisci-Formula-Percezione(*percezione*, *t* ))

*azione*  $\leftarrow$  ASK(*KB*, Costruisci-Query-Azione( *t* ))

TELL(*KB*, Costruisci-Formula-Azione(*azione*, *t* ))

*t*  $\leftarrow$  *t* + 1

**return** *azione*

# Programma di un agente KB

prende in input una percezione e restituisce un'azione

**function** Agente-KB (*percezione*) **returns** *un'azione*

**persistent:** *KB*, una base di conoscenza

*t*, un contatore, inizialmente a 0, che indica il tempo

TELL(*KB*, Costruisci-Formula-Percezione(*percezione*, *t* ))

*azione*  $\leftarrow$  ASK(*KB*, Costruisci-Query-Azione( *t* ))

TELL(*KB*, Costruisci-Formula-Azione(*azione*, *t* ))

*t*  $\leftarrow$  *t* + 1

**return** *azione*

# Programma di un agente KB

**function** Agente-KB (*percezione*) **returns** *un'azione*

**persistent:** *KB*, una base di conoscenza

*t*, un contatore, inizialmente a 0, che indica il tempo

*mantiene in memoria una base di conoscenza KB che puo' contenere una conoscenza iniziale*

TELL(*KB*, Costruisci-Formula-Percezione(*percezione*, *t* ))

*azione*  $\leftarrow$  ASK(*KB*, Costruisci-Query-Azione( *t* ))

TELL(*KB*, Costruisci-Formula-Azione(*azione*, *t* ))

*t*  $\leftarrow$  *t* + 1

**return** *azione*

# Programma di un agente KB

**function** Agente-KB (*percezione*) **returns** *un'azione*

**persistent:** *KB*, una base di conoscenza

*t*, un contatore, inizialmente a 0, che indica il tempo

```
TELL(KB, Costruisci-Formula-Percezione(percezione, t ))
```

```
azione ← ASK(KB, Costruisci-Query-Azione( t ))
```

```
TELL(KB, Costruisci-Formula-Azione(azione, t ))
```

```
t ← t + 1
```

```
return azione
```

Comunica la percezione alla KB

# Programma di un agente KB

**function** Agente-KB (*percezione*) **returns** *un'azione*

**persistent:** *KB*, una base di conoscenza

*t*, un contatore, inizialmente a 0, che indica il tempo

TELL(*KB*, Costruisci-Formula-Percezione(*percezione*, *t* ))

*azione*  $\leftarrow$  ASK(*KB*, Costruisci-Query-Azione( *t* ))

TELL(*KB*, Costruisci-Formula-Azione(*azione*, *t* ))

*t*  $\leftarrow$  *t* + 1

**return** *azione*

Chiede alla KB quale azione deve compiere

# Programma di un agente KB

**function** Agente-KB (*percezione*) **returns** *un'azione*

**persistent:** *KB*, una base di conoscenza

*t*, un contatore, inizialmente a 0, che indica il tempo

TELL(*KB*, Costruisci-Formula-Percezione(*percezione*, *t* ))

*azione*  $\leftarrow$  ASK(*KB*, Costruisci-Query-Azione( *t* ))

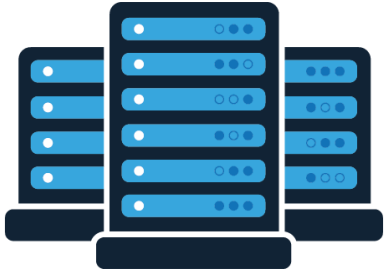
TELL(*KB*, Costruisci-Formula-Azione(*azione*, *t* ))

*t*  $\leftarrow$  *t* + 1

**return** *azione*

comunica alla KB che l'azione e' stata compiuta al tempo *t*

# Base di conoscenza vs base di dati



*Base di dati*: solo fatti specifici, solo recupero (*retrieval*)

*Base di conoscenza*: una rappresentazione esplicita, parziale e compatta, in un *linguaggio simbolico*, che contiene:

- fatti di tipo specifico
  - *Casella [1, 1] ok*
  - *C'è un Pozzo in [3,1]*
- fatti di tipo generale, o regole
  - *C'è brezza nelle caselle adiacenti ai pozzi*



# Base di conoscenza vs base di dati

*Base di conoscenza*: una rappresentazione esplicita, parziale e compatta, in un *linguaggio simbolico*, che contiene:

- fatti di tipo specifico
  - *Casella [1, 1] ok*
  - *C'è un Pozzo in [3,1]*
- fatti di tipo generale, o regole
  - *C'è brezza nelle caselle adiacenti ai pozzi*



Quello che caratterizza una KB è *la capacità inferenziale*

- derivare nuovi fatti da quelli memorizzati esplicitamente

Es. *C'è un Pozzo in [3, 1]*  
*Il Wumpus è in [1,3]*

## Il *trade-off* fondamentale della Rappresentazione della Conoscenza (R.C.)

- Sfortunatamente più il linguaggio è *espressivo*, meno *efficiente* è il meccanismo inferenziale.
- Il problema «fondamentale» in R.C. è trovare il giusto compromesso tra:
  - *Espressività* del linguaggio di rappresentazione
  - *Complessità* del meccanismo inferenziale
- Questi due obiettivi sono in contrasto e si tratta di mediare tra queste due esigenze

Logica

# La Logica

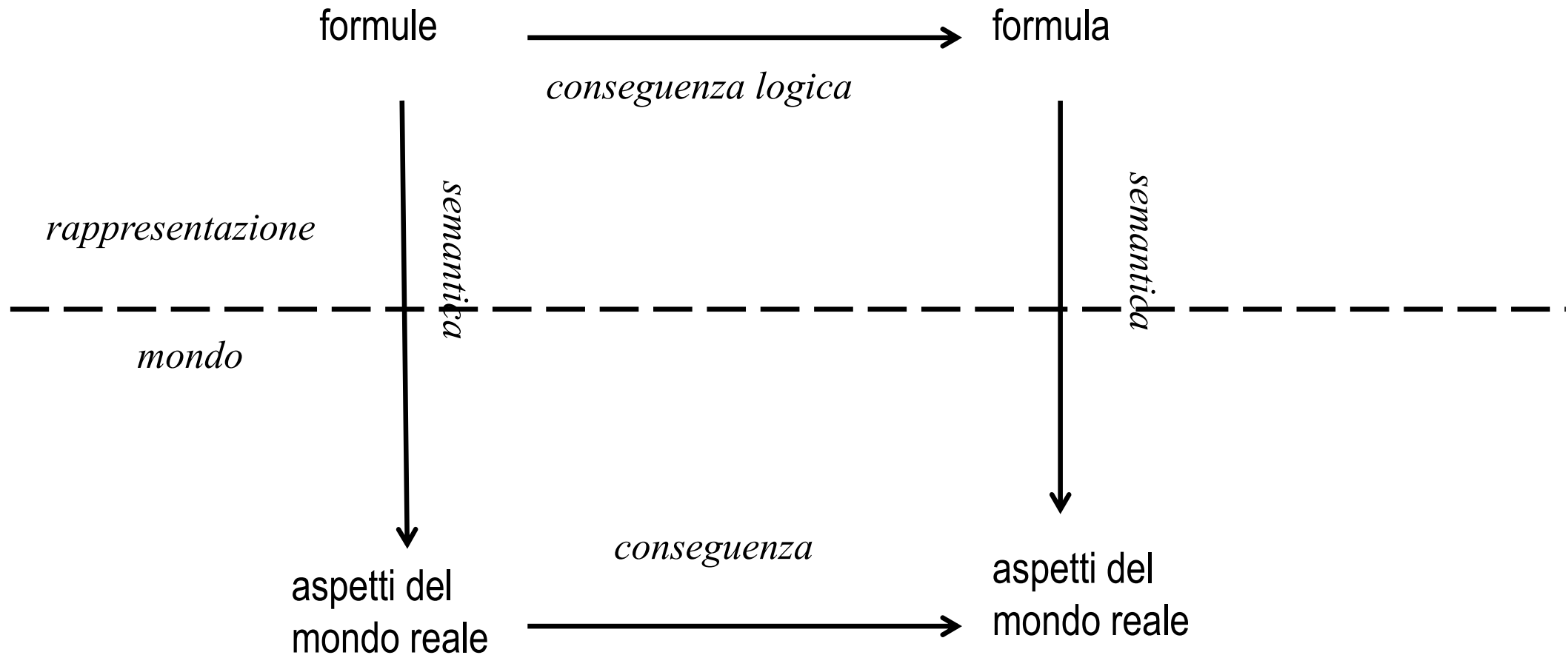
- Le basi di conoscenza sono costituite da enunciati (formule)
- Le formule sono espresse secondo le regole della **sintassi**, che specifica quali di esse sono “ben formate”
- La **semantica** di una formula esprime il “significato” della formula
- Un **modello** e' una configurazione dei valori di verità che si possono assegnare alle variabili coinvolte in una formula

# Formalismi per la Rappresentazione della Conoscenza

Un formalismo per la rappresentazione della conoscenza ha tre componenti:

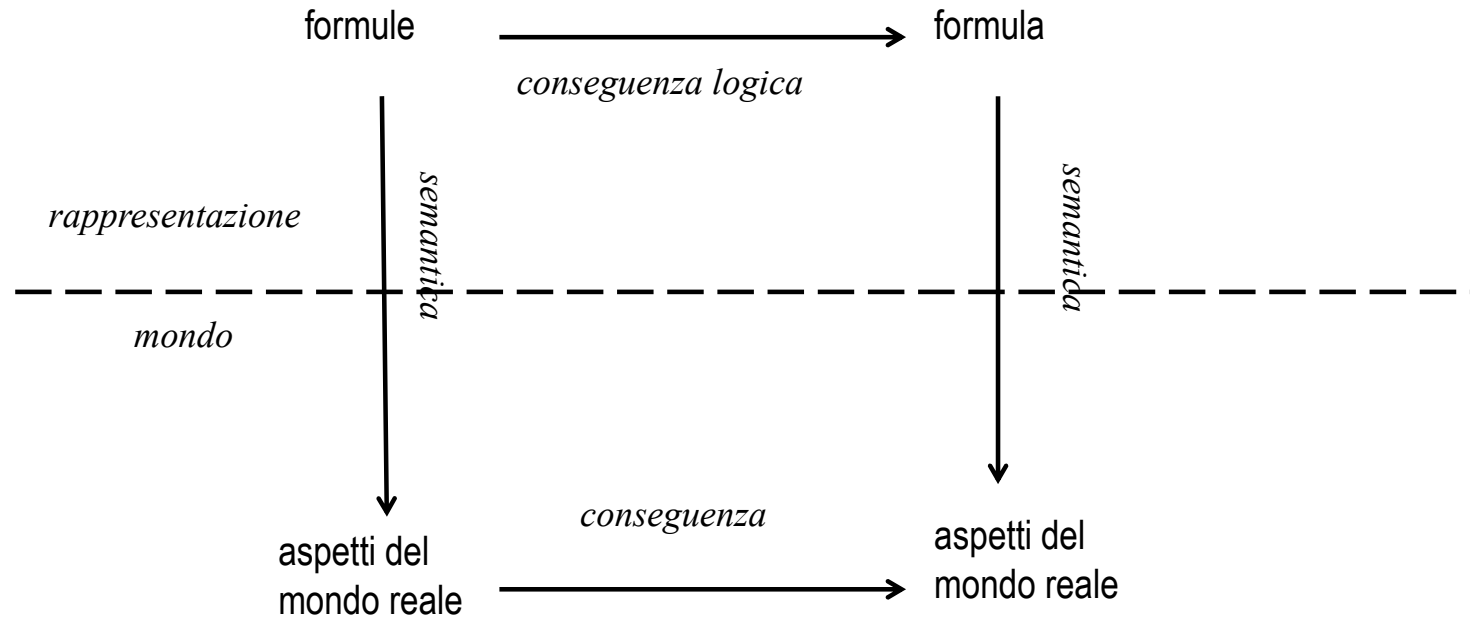
1. Una *sintassi*: un linguaggio composto da un vocabolario e regole per la formazione delle frasi (*enunciati, formule*).
2. Una *semantica*: stabilisce una corrispondenza tra gli enunciati e i fatti del mondo; se un agente ha un enunciato  $\alpha$  nella sua KB, crede che il fatto corrispondente sia vero nel mondo.
3. Un *meccanismo inferenziale* (codificato o meno tramite regole di inferenza come nella logica) che ci consente di inferire nuovi fatti.

# Rappresentazione e mondo



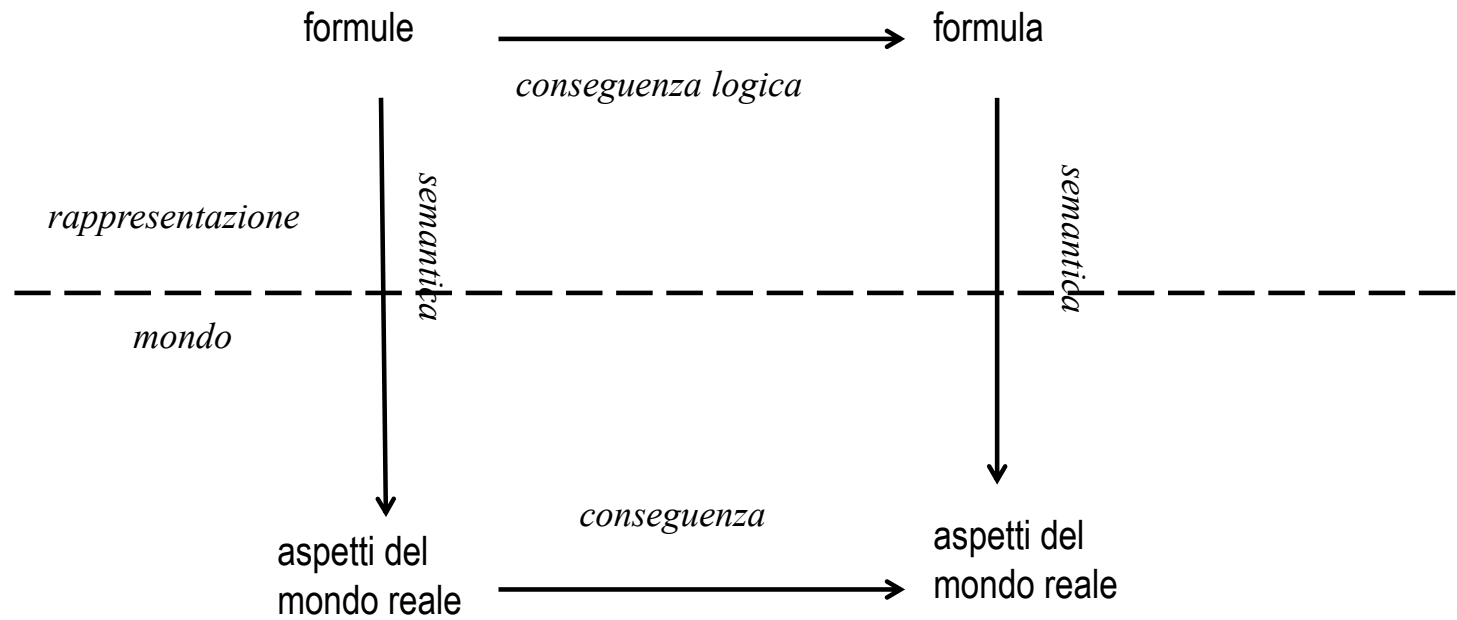
# Rappresentazione e mondo

Le formule sono  
configurazioni fisiche  
dell'agente

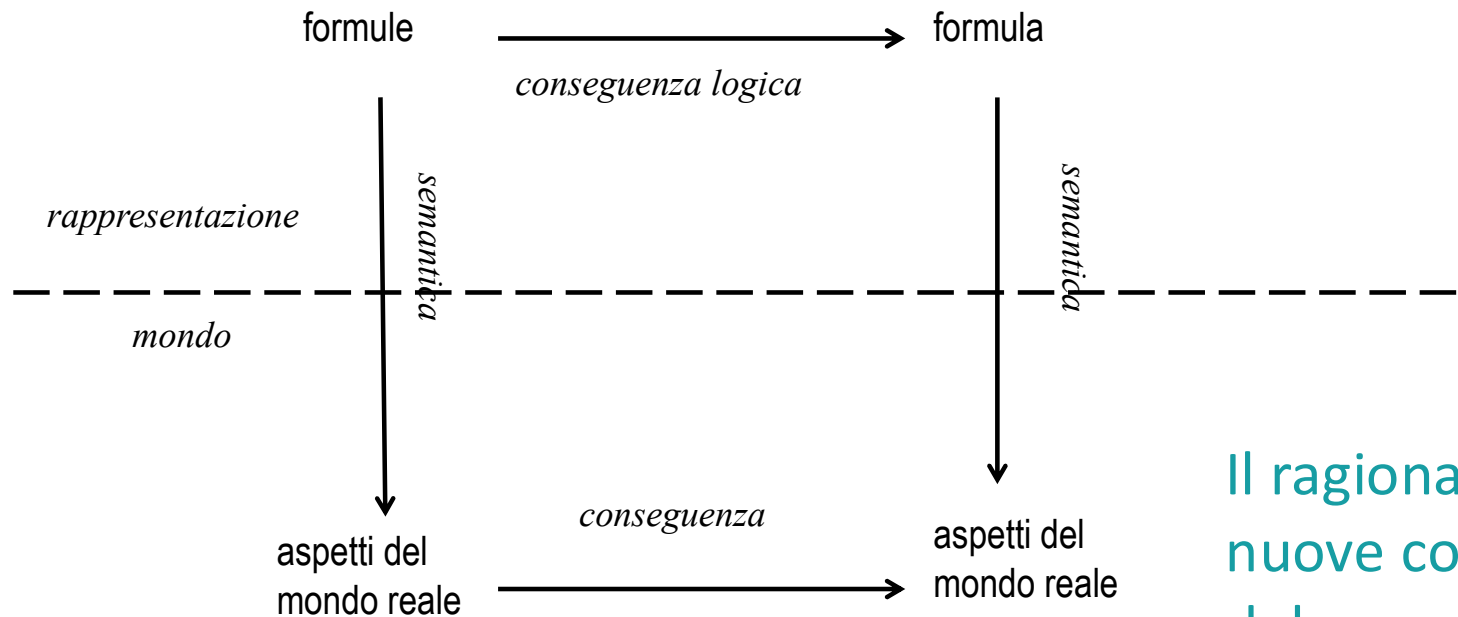


# Rappresentazione e mondo

Il ragionamento è il processo di costruzione di nuove configurazioni partendo dalle vecchie



# Rappresentazione e mondo



Il ragionamento dovrebbe assicurare che le nuove configurazioni rappresentino aspetti del mondo che sono effettive conseguenze degli aspetti del mondo rappresentati dalle vecchie configurazioni

# Logica e Rappresentazione della Conoscenza

- Problema  $KB \models \alpha$
- Soluzione algoritmiche
- Linguaggi logici:
  - Calcolo proposizionale (PROP)
  - Logica dei predicati del prim'ordine (FOL)

# Agenti logici: calcolo proposizionale (logica proposizionale)

*Come costruire agenti  
dotati di capacità  
inferenziali*

Claudio Gallicchio,  
2025/2026

*[claudio.gallicchio@unipi.it](mailto:claudio.gallicchio@unipi.it)*

# Riferimenti



AIMA Cap 7

■ 7.3 & 7.4

# Logica Proposizionale

**Sintassi** La sintassi definisce quali sono le frasi legittime  
(**ben formate**) del linguaggio:

$$formula \rightarrow formulaAtomica \mid formulaComplessa$$
$$formulaAtomica \rightarrow \mathbf{True} \mid \mathbf{False} \mid simbolo$$
$$simbolo \rightarrow \mathbf{P} \mid \mathbf{Q} \mid \mathbf{R} \mid \dots$$
$$formulaComplessa \rightarrow \neg formula$$
$$\mid (formula \wedge formula)$$
$$\mid (formula \vee formula)$$
$$\mid (formula \Rightarrow formula)$$
$$\mid (formula \Leftrightarrow formula)$$

una grammatica delle  
formule della logica  
proposizionale in forma  
BNF (Backus-Naur Form)



**Sintassi** La sintassi definisce quali sono le frasi legittime  
(**ben formate**) del linguaggio:

$formula \rightarrow formulaAtomica \mid formulaComplessa$

$formulaAtomica \rightarrow \mathbf{True} \mid \mathbf{False} \mid simbolo$

$simbolo \rightarrow \mathbf{P} \mid \mathbf{Q} \mid \mathbf{R} \mid \dots$

$formulaComplessa \rightarrow \neg formula$  not (negazione)

and (congiunzione)  $\mid (formula \wedge formula)$

or (disgiunzione)  $\mid (formula \vee formula)$

implicazione  $\mid (formula \Rightarrow formula)$

se e solo se  $\mid (formula \Leftrightarrow formula)$

operatori chiamati  
**connettivi logici**

# Sintassi: esempi

- $((A \wedge B) \Rightarrow C)$

Possiamo omettere le parentesi assumendo questa **precedenza** tra gli operatori:

$$\neg > \wedge > \vee > \Rightarrow, \Leftrightarrow$$

- $\neg P \vee Q \wedge R \Rightarrow S$  è la stessa cosa di  $((\neg P) \vee (Q \wedge R)) \Rightarrow S$
- Es. dal mondo del Wumpus
  - $P_{1,1}$  c'è un pozzo in  $[1,1]$
  - $W_{2,3}$  il Wumpus è in  $[2,3]$

# Semantica

La semantica ha a che fare col significato delle frasi: definisce se un enunciato è vero o falso rispetto a una *interpretazione* (mondo possibile)

Nella logica proposizionale, una interpretazione specifica il valore di verità (True o False) di ogni simbolo proposizionale

# Semantica e mondi possibili (modelli)

- Una interpretazione definisce un valore di verità per tutti i simboli proposizionali.
  - Esempio:  $\{P_{1,1} \text{ vero}, P_{1,2} \text{ falso}, W_{2,3} \text{ vero}\}$
- Il valore di una formula complessa è fissato di conseguenza
  - $P_{1,1} \Rightarrow W_{2,3} \vee P_{1,2}$  è *vera* in questa interpretazione
- Un *modello* di una formula (resp. insieme di formule) è una interpretazione *che la rende vera* (resp. che rende vere *tutte* le formule dell'insieme)

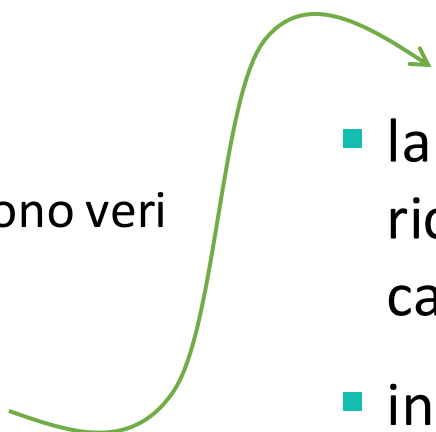
# Semantica composizionale

Il significato di una frase è determinato dal significato dei suoi componenti, a partire dalle frasi atomiche (i simboli proposizionali)

- *True* sempre vero; *False* sempre falso
- $P \wedge Q$ , vero se P e Q sono entrambi veri
- $P \vee Q$ , vero se P oppure Q, o entrambi, sono veri
- $\neg P$  vero se P è falso
- $P \Rightarrow Q$ , vero se P è falso oppure Q è vero
- $P \Leftrightarrow Q$ , vero se entrambi veri o entrambi falsi

# Semantica composizionale

Il significato di una frase è determinato dal significato dei suoi componenti, a partire dalle frasi atomiche (i simboli proposizionali)

- *True* sempre vero; *False* sempre falso
  - $P \wedge Q$ , vero se P e Q sono entrambi veri
  - $P \vee Q$ , vero se P oppure Q, o entrambi, sono veri
  - $\neg P$  vero se P è falso
  - $P \Rightarrow Q$ , vero se P è falso oppure Q è vero
  - $P \Leftrightarrow Q$ , vero se entrambi veri o entrambi falsi
- 
- la logica proposizionale non richiede alcuna relazione di causalità
  - in effetti significa che “Se P è vero, allora sostengo che lo è anche Q. Se P è falso, allora non faccio alcuna asserzione”
    - P e Q sono entrambi veri oppure P è falso.

# Tavola di verità

Specifica il valore di verità di una formula complessa per ogni possibile configurazione dei suoi componenti

| $P$          | $Q$          | $\neg P$     | $P \wedge Q$ | $P \vee Q$   | $P \Rightarrow Q$ | $P \Leftrightarrow Q$ |
|--------------|--------------|--------------|--------------|--------------|-------------------|-----------------------|
| <i>false</i> | <i>false</i> | <i>true</i>  | <i>false</i> | <i>false</i> | <i>true</i>       | <i>true</i>           |
| <i>false</i> | <i>true</i>  | <i>true</i>  | <i>false</i> | <i>true</i>  | <i>true</i>       | <i>false</i>          |
| <i>true</i>  | <i>false</i> | <i>false</i> | <i>false</i> | <i>true</i>  | <i>false</i>      | <i>false</i>          |
| <i>true</i>  | <i>true</i>  | <i>false</i> | <i>true</i>  | <i>true</i>  | <i>true</i>       | <i>true</i>           |

# Conseguenza logica



Una formula  $\alpha$  è *conseguenza logica* di un insieme di formule KB se e solo se in ogni modello di KB, anche  $\alpha$  è vera ( $KB \models \alpha$ )

Indicando con  $M(KB)$  i modelli dell'insieme di formule in KB e con  $M(\alpha)$  l'insieme delle interpretazioni che rendono  $\alpha$  vera (i *modelli* di  $\alpha$ ) ...

$$KB \models \alpha \text{ sse } M(KB) \subseteq M(\alpha)$$

# Model checking

Un modo (semplice) per determinare la conseguenza logica

- Si enumerano i modelli
- Si mostra che la formula  $\alpha$  deve valere in tutti i modelli in cui è vera la KB
- Si usa per eseguire “inferenze logiche”
  - derivare una conclusione che segue logicamente da una KB

# Esempio dal mondo del Wumpus (1)

La KB iniziale  $KB_0$  è costituita dalle regole generali del WW e dal fatto che la casella iniziale è *safe* per definizione

$$\neg W_{1,1} \quad \neg P_{1,1}$$

$$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

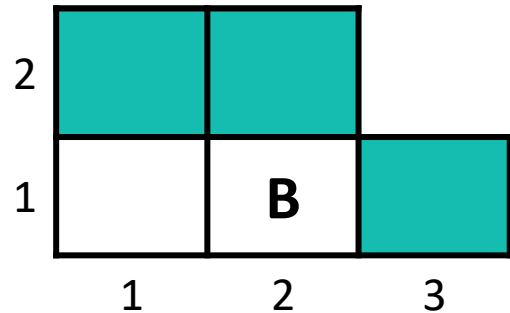
$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

...

La casella iniziale è *safe*

Esempi di regole generali

## Esempio dal mondo del Wumpus (2)



L'agente non ha percepito nulla in [1,1] e si è spostato in [2,1] dove ha percepito una *Brezza*

- $KB_1$  a questo punto è  $KB_0$  più i *fatti* corrispondenti alle percezioni  
 $KB_0 \cup \{\neg B_{1,1}, B_{2,1}, \neg F_{1,1}, \neg F_{2,1}, \dots\}$
- Domandiamoci se le stanze adiacenti non contengono *Pozzi*

$$KB_1 \models \neg P_{1,2}$$

$$KB_1 \models \neg P_{2,2}$$

$$KB_1 \models \neg P_{3,1}$$



# Esempio dal mondo del Wumpus (3)

Cosa sappiamo all'inizio (quindi cosa c'è in  $KB_0$ ):

- In  $[1,1]$  non ci sono pozzi:
  - $R1: \neg P_{1,1}$
- In una stanza si percepisce brezza se e solo se c'è un pozzo in una stanza adiacente (di seguito solo per le posizioni rilevanti):
  - $R2: B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
  - $R3: B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$

# Esempio dal mondo del Wumpus (4)

Cosa sappiamo in base alle percezioni (cosa aggiungiamo alla KB):

- Non c'è brezza in [1,1]:
  - R4:  $\neg B_{1,1}$
- C'è brezza in [2,1]:
  - R5:  $B_{2,1}$

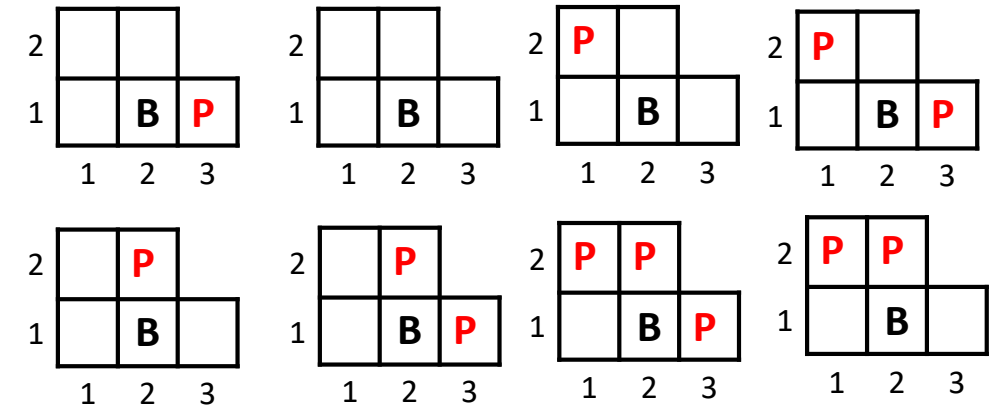
# Esempio dal mondo del Wumpus (5)

| $B_{1,1}$ | $B_{2,1}$ | $P_{1,1}$ | $P_{1,2}$ | $P_{2,1}$ | $P_{2,2}$ | $P_{3,1}$ | $R_1$    | $R_2$    | $R_3$    | $R_4$    | $R_5$    | KB          |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|----------|----------|----------|----------|----------|-------------|
| false     | false     | false     | false     | false     | false     | false     | true     | true     | true     | true     | false    | false       |
| false     | false     | false     | false     | false     | false     | true      | true     | true     | false    | true     | false    | false       |
| $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$    |
| false     | true      | false     | false     | false     | false     | false     | true     | true     | false    | true     | true     | false       |
| false     | true      | false     | false     | false     | false     | true      | true     | true     | true     | true     | true     | <u>true</u> |
| false     | true      | false     | false     | false     | true      | false     | true     | true     | true     | true     | true     | <u>true</u> |
| false     | true      | false     | false     | false     | true      | true      | true     | true     | true     | true     | true     | <u>true</u> |
| false     | true      | false     | false     | true      | false     | false     | true     | false    | false    | true     | true     | false       |
| $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$    |
| true      | true      | true      | true      | true      | true      | true      | false    | true     | true     | false    | true     | false       |

# Esempio dal mondo del Wumpus (6)

| $B_{1,1}$ | $B_{2,1}$ | $P_{1,1}$ | $P_{1,2}$ | $P_{2,1}$ | $P_{2,2}$ | $P_{3,1}$ | $R_1$    | $R_2$    | $R_3$    | $R_4$    | $R_5$    | KB          |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|----------|----------|----------|----------|----------|-------------|
| false     | false     | false     | false     | false     | false     | false     | true     | true     | true     | true     | false    | false       |
| false     | false     | false     | false     | false     | false     | true      | true     | true     | false    | true     | false    | false       |
| $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$    |
| false     | true      | false     | false     | false     | false     | false     | true     | true     | false    | true     | true     | false       |
| false     | true      | false     | false     | false     | false     | true      | true     | true     | true     | true     | true     | <u>true</u> |
| false     | true      | false     | false     | false     | true      | true      | true     | true     | true     | true     | true     | <u>true</u> |
| false     | true      | false     | false     | true      | false     | false     | true     | false    | false    | true     | true     | false       |
| $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$    |
| true      | true      | true      | true      | true      | true      | true      | false    | true     | true     | false    | true     | false       |

Considerando solo l'esistenza di pozzi nelle 3 caselle  $P_{1,2}$ ,  $P_{2,2}$  e  $P_{3,1}$ , ci sono  $2^3=8$  possibili interpretazioni o mondi possibili

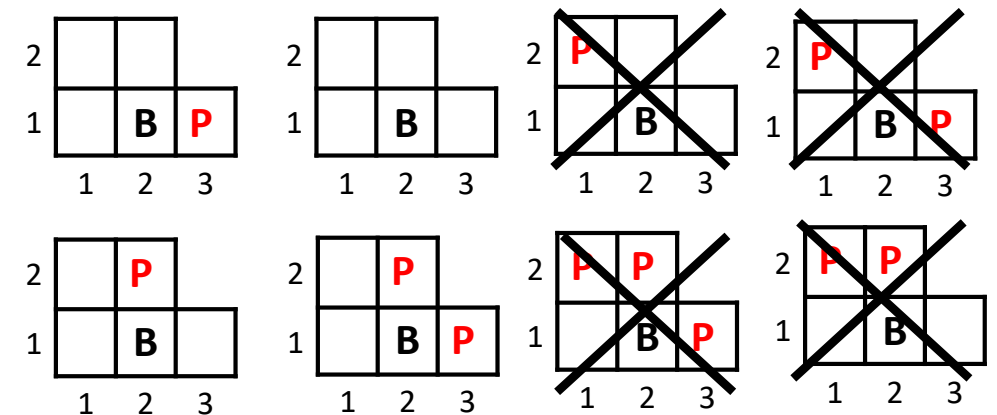


# Esempio dal mondo del Wumpus (7)

| $B_{1,1}$ | $B_{2,1}$ | $P_{1,1}$ | $P_{1,2}$ | $P_{2,1}$ | $P_{2,2}$ | $P_{3,1}$ | $R_1$    | $R_2$    | $R_3$    | $R_4$    | $R_5$    | KB       |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|----------|----------|----------|----------|----------|----------|
| false     | false     | false     | false     | false     | false     | false     | true     | true     | true     | true     | false    | false    |
| false     | false     | false     | false     | false     | false     | true      | true     | true     | false    | true     | false    | false    |
| $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |
| false     | true      | false     | false     | false     | false     | false     | true     | true     | false    | true     | true     | false    |
| false     | true      | false     | false     | false     | false     | true      | true     | true     | true     | true     | true     | true     |
| false     | true      | false     | false     | false     | true      | false     | true     | true     | true     | true     | true     | true     |
| false     | true      | false     | false     | false     | true      | true      | true     | true     | true     | true     | true     | true     |
| false     | true      | false     | false     | true      | false     | false     | true     | false    | false    | true     | true     | false    |
| $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$  | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |
| true      | true      | true      | true      | true      | true      | true      | false    | true     | true     | false    | true     | false    |

$$KB_1 \models \neg P_{1,2}$$

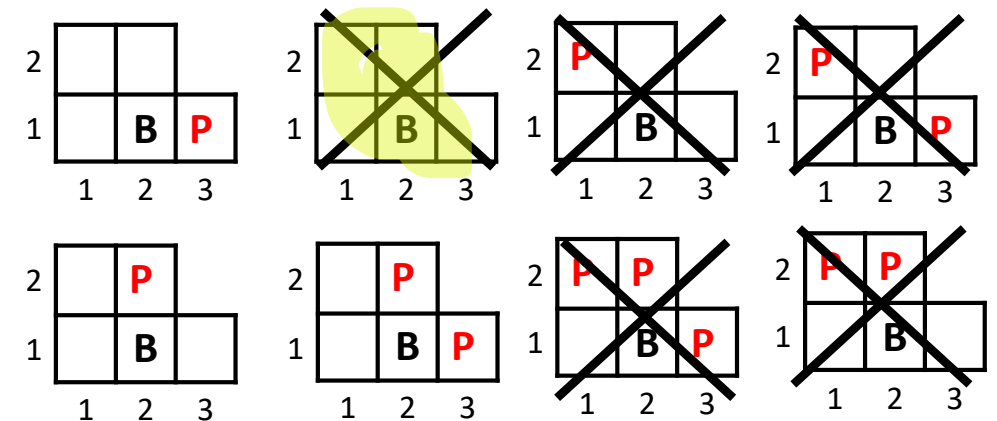
Considerando solo l'esistenza di pozzi nelle 3 caselle  $P_{1,2}$ ,  $P_{2,2}$  e  $P_{3,1}$ , ci sono  $2^3=8$  possibili interpretazioni o mondi possibili



# Esempio dal mondo del Wumpus (8)

| $B_{1,1}$ | $B_{2,1}$ | $P_{1,1}$ | $P_{1,2}$ | $P_{2,1}$ | $P_{2,2}$ | $P_{3,1}$ | $R_1$ | $R_2$ | $R_3$ | $R_4$ | $R_5$ | KB    |
|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-------|-------|-------|-------|-------|-------|
| false     | false     | false     | false     | false     | false     | false     | true  | true  | true  | true  | false | false |
| false     | false     | false     | false     | false     | false     | true      | true  | true  | false | true  | false | false |
| ⋮         | ⋮         | ⋮         | ⋮         | ⋮         | ⋮         | ⋮         | ⋮     | ⋮     | ⋮     | ⋮     | ⋮     | ⋮     |
| false     | true      | false     | false     | false     | false     | false     | true  | true  | false | true  | true  | false |
| false     | true      | false     | false     | false     | false     | true      | true  | true  | true  | true  | true  | true  |
| false     | true      | false     | false     | false     | false     | false     | true  | true  | true  | true  | true  | true  |
| false     | true      | false     | false     | false     | false     | true      | true  | true  | true  | true  | true  | true  |
| false     | true      | false     | false     | true      | false     | false     | true  | false | false | true  | true  | false |
| ⋮         | ⋮         | ⋮         | ⋮         | ⋮         | ⋮         | ⋮         | ⋮     | ⋮     | ⋮     | ⋮     | ⋮     | ⋮     |
| true      | true      | true      | true      | true      | true      | true      | false | true  | true  | false | true  | false |

Considerando solo l'esistenza di pozzi nelle 3 caselle  $P_{1,2}$ ,  $P_{2,2}$  e  $P_{3,1}$ , ci sono  $2^3=8$  possibili interpretazioni o mondi possibili



$$KB_1 \models \neg P_{1,2}$$

$$KB_1 \models P_{2,2} \vee P_{3,1}$$

Nei modelli di  $KB_1$  nulla possiamo concludere su  $\neg P_{2,2}$  e  $\neg P_{3,1}$  (né su  $P_{2,2}$  e  $P_{3,1}$  ...)

# Agenti logici: calcolo proposizionale

*Claudio Gallicchio*

*a.a. 2025/2026*

# Riferimenti



AIMA Cap 7

■ 7.6

Dimostrazione di teoremi

# Dimostrazione di teoremi nella logica proposizionale

- Fin qui abbiamo visto come determinare la **conseguenza logica tramite il model checking**
  - enumerare i modelli e mostrare che la formula deve valere in tutti
- La conseguenza logica puo' essere ottenuta anche tramite la **dimostrazione di teoremi**
  - **applicando regole di inferenza** direttamente alle formule della KB
  - costruire una dimostrazione della formula desiderata **senza consultare i modelli**

# Dimostrazione di teoremi nella logica proposizionale

Tre fondamentali concetti relativi alla conseguenza logica:

1. Equivalenza logica
2. Validità
3. Soddisfacibilità



# Equivalenza logica

- Due formule  $\alpha$  e  $\beta$  sono equivalenti se sono vere nello stesso insieme di modelli
- $\alpha \equiv \beta$  se e solo se  $\alpha \models \beta$  e  $\beta \models \alpha$

Esempi:

$$A \wedge B \equiv B \wedge A$$

$$\neg(A \wedge B) \equiv \neg A \vee \neg B$$

$$(A \wedge (A \vee B)) \equiv A$$



*Il ruolo di queste equivalenze è analogo a quello delle identità aritmetiche in matematica.*

## Equivalenze logiche (leggi)

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$$

$$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$$

$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$$

$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$$

$$\neg(\neg\alpha) \equiv \alpha$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$$

$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$$

commutatività di  $\wedge$

commutatività di  $\vee$

associatività di  $\wedge$

associatività di  $\vee$

eliminazione della doppia negazione

contrapposizione

eliminazione dell'implicazione

eliminazione del bicondizionale

De Morgan

De Morgan

distributività di  $\wedge$  su  $\vee$

distributività di  $\vee$  su  $\wedge$





# Validità

- Una formula  $\alpha$  è **valida** sse è vera in tutte le interpretazioni
  - Es.  $A \vee \neg A$
- Le formule valide sono anche dette **tautologie**
  - sono formule necessariamente vere
- **Teorema di deduzione**
  - date due formule  $\alpha$  e  $\beta$ :  $\alpha \models \beta$  se e solo se  $(\alpha \Rightarrow \beta)$  è valida



# Soddisfacibilità

- Una formula  $\alpha$  è **soddisfacibile** sse esiste una interpretazione in cui  $\alpha$  è vera (ovvero se esiste un **modello** di  $\alpha$ )
  - SAT: determinare la soddisfacibilità di formule della logica proposizionale
- Validità e soddisfacibilità sono strettamente connesse
  - $\alpha$  è **valida** sse  $\neg\alpha$  è **insoddisfacibile**
  - $\alpha$  è **soddisfacibile** sse  $\neg\alpha$  non è **valida**

# Dimostrazione per assurdo

- Th. di deduzione:  $\alpha \models \beta$  se e solo se  $(\alpha \Rightarrow \beta)$  è valida
  - $\alpha \models \beta$  se e solo se  $\neg(\alpha \Rightarrow \beta)$  è insoddisfacibile
  - $(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$  [eliminazione dell'implicazione]
  - $\neg(\alpha \Rightarrow \beta) \equiv \neg(\neg\alpha \vee \beta) \equiv (\alpha \wedge \neg\beta)$  [De Morgan]
  - $\alpha \models \beta$  se e solo se  $(\alpha \wedge \neg\beta)$  è insoddisfacibile
    - dimostrazione per refutazione o per contraddizione
    - si assume che  $\beta$  sia falsa e si dimostra che questo porta a una contraddizione con gli assiomi  $\alpha$
-

# Inferenza per PROP

## ■ *Model checking*

- una forma di inferenza che fa riferimento alla definizione di conseguenza logica (si enumerano i possibili modelli)
- Tecnica delle tabelle di verità (TV)

## ■ Algoritmi per la *soddisfacibilità* (SAT)

- $KB \models \alpha$  sse  $(KB \wedge \neg\alpha)$  è insoddisfacibile (*Teorema di refutazione*)
  - ✓ dimostrare  $\alpha$  partendo da KB verificando che  $(KB \wedge \neg\alpha)$  non è mai vero (dimostrazione per assurdo)
  - ✓ si parte da  $\neg\alpha$  e si dimostra che questo porta a una contraddizione con gli assiomi in KB (che sono noti)
- La conseguenza logica può essere ricondotta a un problema SAT

Model checking

# Model checking

Consiste in:

- Enumerare tutti i possibili modelli di KB
  - ✓ tramite il calcolo della tavola di verità (TV)
- Verificare che in essi la formula  $\alpha$  sia vera

$$M(KB) \subseteq M(\alpha)$$



# L'algoritmo TV-Consegue?

- Quesito:  $\alpha$  è conseguenza logica di KB? ( $KB \models \alpha$  ?)
- Enumera tutte le possibili interpretazioni di KB
  - ( $k$  simboli,  $2^k$  possibili interpretazioni)
- Per ciascuna interpretazione
  - Se non soddisfa KB, OK
  - Se soddisfa KB, si controlla che soddisfi anche  $\alpha$
- Se si trova anche solo una interpretazione che soddisfa KB e non  $\alpha$  la risposta sarà NO.

# Funzione TV-Consegue?

```
function TV-Consegue?( $KB, \alpha$ ) // returns true oppure false  
    inputs:  $KB$ , la base di conoscenza, una formula della logica proposizionale  
             $\alpha$ , la query, una formula della logica proposizionale  
  
     $simboli \leftarrow$  una lista dei simboli proposizionali contenuti in  $KB$  e  $\alpha$   
    return TV-Verifica-Tutto( $KB, \alpha, simboli, \{ \}$ )
```

```
function TV-Verifica-Tutto( $KB, \alpha, simboli, modello$ ) // returns true oppure false  
if Vuoto?( $simboli$ ) then  
    if PL-Vero?( $KB, modello$ ) then return PL-Vero?( $\alpha, modello$ )  
    else return true // quando  $KB$  è false, restituisce sempre true  
else do  
     $P \leftarrow$  Primo( $simboli$ );  $resto \leftarrow$  Resto( $simboli$ )  
    return TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = true\}$ )  
        and  
        TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = false\}$ )
```

# Funzione TV-Consegue?

**function TV-Consegue?**( $KB, \alpha$ ) // returns *true* oppure *false*

**inputs:**  $KB$ , la base di conoscenza, una formula della logica proposizionale  
 $\alpha$ , la query, una formula della logica proposizionale

$simboli \leftarrow$  una lista dei simboli proposizionali contenuti in  $KB$  e  $\alpha$

**return** TV-Verifica-Tutto( $KB, \alpha, simboli, \{ \}$ )

**function TV-Verifica-Tutto**( $KB, \alpha, simboli, modello$ ) //returns *true* oppure *false*

**if** Vuoto?( $simboli$ ) **then**

**if** PL-Vero?( $KB, modello$ ) **then return** PL-Vero?( $\alpha, modello$ )

**else return** *true* // quando  $KB$  è *false*, restituisce sempre *true*

**else do**

$P \leftarrow$  Primo( $simboli$ );  $resto \leftarrow$  Resto( $simboli$ )

**return** TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = true\}$ )

**and**

        TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = false\}$ )

# Funzione TV-Consegue?

```
function TV-Consegue?( $KB, \alpha$ ) // returns true oppure false  
    inputs:  $KB$ , la base di conoscenza, una formula della logica proposizionale  
             $\alpha$ , la query, una formula della logica proposizionale
```

```
 $simboli \leftarrow$  una lista dei simboli proposizionali contenuti in  $KB$  e  $\alpha$ 
```

```
return TV-Verifica-Tutto( $KB, \alpha, simboli, \{ \}$ )
```

```
function TV-Verifica-Tutto( $KB, \alpha, simboli, modello$ ) // returns true oppure false
```

```
if Vuoto?( $simboli$ ) then
```

```
    if PL-Vero?( $KB, modello$ ) then return PL-Vero?( $\alpha, modello$ )
```

```
    else return true // quando  $KB$  è false, restituisce sempre true
```

```
else do
```

```
     $P \leftarrow$  Primo( $simboli$ );  $resto \leftarrow$  Resto( $simboli$ )
```

```
    return TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = true\}$ )
```

```
        and
```

```
        TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = false\}$ )
```

# Funzione TV-Consegue?

```
function TV-Consegue?( $KB, \alpha$ ) // returns true oppure false  
    inputs:  $KB$ , la base di conoscenza, una formula della logica proposizionale  
             $\alpha$ , la query, una formula della logica proposizionale  
  
     $simboli \leftarrow$  una lista dei simboli proposizionali contenuti in  $KB$  e  $\alpha$   
return TV-Verifica-Tutto( $KB, \alpha, simboli, \{ \}$ )
```

```
function TV-Verifica-Tutto( $KB, \alpha, simboli, modello$ ) // returns true oppure false  
if Vuoto?( $simboli$ ) then  
    if PL-Vero?( $KB, modello$ ) then return PL-Vero?( $\alpha, modello$ )  
    else return true // quando  $KB$  è false, restituisce sempre true  
else do  
     $P \leftarrow$  Primo( $simboli$ );  $resto \leftarrow$  Resto( $simboli$ )  
    return TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = true\}$ )  
        and  
        TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = false\}$ )
```

# Funzione TV-Consegue?

**function TV-Consegue?**( $KB, \alpha$ ) // returns *true* oppure *false*

**inputs:**  $KB$ , la base di conoscenza, una formula della logica proposizionale  
 $\alpha$ , la query, una formula della logica proposizionale

$simboli \leftarrow$  una lista dei simboli proposizionali contenuti in  $KB$  e  $\alpha$

**return** TV-Verifica-Tutto( $KB, \alpha, simboli, \{\}$ ) un modello parziale, all'inizio vuoto

**function TV-Verifica-Tutto**( $KB, \alpha, simboli, modello$ ) //returns *true* oppure *false*

**if** Vuoto?( $simboli$ ) **then**

**if** PL-Vero?( $KB, modello$ ) **then return** PL-Vero?( $\alpha, modello$ )

**else return true** // quando  $KB$  è *false*, restituisce sempre *true*

**else do**

$P \leftarrow$  Primo( $simboli$ );  $resto \leftarrow$  Resto( $simboli$ )

**return** TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = true\}$ )

**and**

        TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = false\}$ )

# Funzione TV-Consegue?

Verifica se, dato il modello, KB e  $\alpha$  sono entrambe vere, oppure KB è falsa

```
function TV-Consegue?(KB,  $\alpha$ ) // returns true oppure false  
    inputs: KB, la base di conoscenza, una formula della logica proposizionale  
             $\alpha$ , la query, una formula della logica proposizionale  
  
    simboli  $\leftarrow$  una lista dei simboli proposizionali contenuti in KB e  $\alpha$   
    return TV-Verifica-Tutto(KB,  $\alpha$ , simboli, { })
```

```
function TV-Verifica-Tutto(KB,  $\alpha$ , simboli, modello) // returns true oppure false
```

```
    if Vuoto?(simboli) then  
        if PL-Vero?(KB, modello) then return PL-Vero?( $\alpha$ , modello)  
        else return true // quando KB è false, restituisce sempre true  
    else do  
        P  $\leftarrow$  Primo(simboli); resto  $\leftarrow$  Resto(simboli)  
        return TV-Verifica-Tutto(KB,  $\alpha$ , resto, modello  $\leftarrow$  {P = true})  
            and  
            TV-Verifica-Tutto(KB,  $\alpha$ , resto, modello  $\leftarrow$  {P = false})
```

*modello* rappresenta un modello parziale (in cui sono stati assegnati valori solo ad alcune variabili)

# Funzione TV-Consegue?

PL-Vero( $KB, model$ ) restituisce True se  $KB$  è vera nella interpretazione  $model$ .

```
function TV-Consegue?( $KB, \alpha$ ) // returns true oppure false  
    inputs:  $KB$ , la base di conoscenza, una formula della logica proposizionale  
             $\alpha$ , la query, una formula della logica proposizionale  
  
     $simboli \leftarrow$  una lista dei simboli proposizionali contenuti in  $KB$  e  $\alpha$   
    return TV-Verifica-Tutto( $KB, \alpha, simboli, \{ \}$ )
```

```
function TV-Verifica-Tutto( $KB, \alpha, simboli, modello$ ) // returns true oppure false
```

```
    if Vuoto?( $simboli$ ) then
```

```
        if PL-Vero?( $KB, modello$ ) then return PL-Vero?( $\alpha, modello$ )
```

```
        else return true // quando  $KB$  è false, restituisce sempre true
```

```
    else do
```

```
         $P \leftarrow$  Primo( $simboli$ );  $resto \leftarrow$  Resto( $simboli$ )
```

```
        return TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = true\}$ )
```

```
            and
```

```
            TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = false\}$ )
```

# Funzione TV-Consegue?

```
function TV-Consegue?( $KB, \alpha$ ) // returns true oppure false  
    inputs:  $KB$ , la base di conoscenza, una formula della logica proposizionale  
             $\alpha$ , la query, una formula della logica proposizionale  
  
     $simboli \leftarrow$  una lista dei simboli proposizionali contenuti in  $KB$  e  $\alpha$   
    return TV-Verifica-Tutto( $KB, \alpha, simboli, \{ \}$ )
```

```
function TV-Verifica-Tutto( $KB, \alpha, simboli, modello$ ) // returns true oppure false  
if Vuoto?( $simboli$ ) then  
    if PL-Vero?( $KB, modello$ ) then return PL-Vero?( $\alpha, modello$ )  
    else return true // quando  $KB$  è false. restituisce sempre true  
  
else do  
     $P \leftarrow$  Primo( $simboli$ );  $resto \leftarrow$  Resto( $simboli$ )  
    return TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = true\}$ )  
        and  
        TV-Verifica-Tutto( $KB, \alpha, resto, modello \leftarrow \{P = false\}$ )
```

# Tabella di verità

$$\overbrace{(\neg a \vee b) \wedge (a \vee c)}^{\text{KB}} \models \overbrace{(b \vee c)}^{\text{formula}}$$



| a | b | c | $\neg a \vee b$ | $a \vee c$ | $b \vee c$ |
|---|---|---|-----------------|------------|------------|
| T | T | T | T               | T          | T          |
| T | T | F | T               | T          | T          |
|   |   |   |                 |            |            |
| F | T | T | T               | T          | T          |
|   |   |   |                 |            |            |
| F | F | T | T               | T          | T          |
|   |   |   |                 |            |            |

individuo le  
interpretazioni che  
sono modelli di KB

verifico se la  
formula è vera in  
tutti i modelli

# Esempio di TV-Consegue?

KB:  $(\neg a \vee b) \wedge (a \vee c)$

$\alpha$ :  $(b \vee c)$

*simboli* =  $[a, b, c]$

## ■ TV-VERIFICA-TUTTO(KB, $\alpha$ , $[a, b, c]$ , $\{ \}$ )

### ■ TV-VERIFICA-TUTTO(KB, $\alpha$ , $[b, c]$ , $\{a=T\}$ )

#### ✓ TV-VERIFICA-TUTTO(KB, $\alpha$ , $[c]$ , $\{a=T, b=T\}$ )

- TV-VERIFICA-TUTTO(KB,  $\alpha$ ,  $[ ]$ ,  $\{a=T, b=T, c=T\}$ ) OK
- TV-VERIFICA-TUTTO(KB,  $\alpha$ ,  $[ ]$ ,  $\{a=T, b=T, c=F\}$ ) OK

#### ✓ TV-VERIFICA-TUTTO(KB, $\alpha$ , $[c]$ , $\{a=T, b=F\}$ )

- TV-VERIFICA-TUTTO(KB,  $\alpha$ ,  $[ ]$ ,  $\{a=T, b=F, c=T\}$ ) OK
- TV-VERIFICA-TUTTO(KB,  $\alpha$ ,  $[ ]$ ,  $\{a=T, b=F, c=F\}$ ) OK

### ■ TV-VERIFICA-TUTTO(KB, $\alpha$ , $[b, c]$ , $\{a=F\}$ ) ...

In questi due casi  
l'interpretazione è un modello  
di KB e di  $\alpha$   
PL-Vero?(KB, *modello*)=true  
PL-Vero?( $\alpha$ , *modello*)=true

In questi due casi  
l'interpretazione non è un  
modello di KB  
PL-Vero?(KB, *modello*)=false

Solo alla fine, dopo avere provato tutti i possibili assegnamenti (interpretazioni)  
possiamo rispondere se  $(b \vee c)$  è o meno conseguenza logica.



# Algoritmi per la soddisfacibilità (SAT)

- Ne vediamo alcuni che usano KB in forma a clausole
- La forma a clausole è la forma normale congiuntiva (CNF): una congiunzione di disgiunzioni di letterali

$$(A \vee B) \wedge (\neg B \vee C \vee D) \wedge (\neg A \vee F)$$

- Non è restrittiva: è sempre possibile ottenerla con trasformazioni che preservano l'equivalenza logica
- Può essere rappresentata come insiemi di letterali

$$\{A, B\} \{\neg B, C, D\} \{\neg A, F\}$$

# Clausole

- Una clausola è una disgiunzione di letterali
- Un **letterale** è dato
  - da un **simbolo** (es. A)
  - dalla **negazione di un simbolo** (es.  $\neg A$ )

Es.

$$A \vee B \vee \neg C \vee D$$

$$\neg A \vee B \vee C \vee \neg D$$

# Trasformazione in forma a clausole

I passi sono:

1. Eliminazione del  $\Leftrightarrow$ :  $(A \Leftrightarrow B) \equiv (A \Rightarrow B) \wedge (B \Rightarrow A)$
2. Eliminazione dell'  $\Rightarrow$ :  $(A \Rightarrow B) \equiv (\neg A \vee B)$
3. Negazioni all'interno:  
 $\neg(A \vee B) \equiv (\neg A \wedge \neg B)$   
 $\neg(A \wedge B) \equiv (\neg A \vee \neg B)$   
*(De Morgan)*
4. Distribuzione di  $\vee$  su  $\wedge$ :  
 $(A \vee (B \wedge C)) \equiv (A \vee B) \wedge (A \vee C)$

# Esempio di trasformazione

*C'è brezza in [1,1] sse c'è un pozzo in [1,2] o [2,1]*

1.  $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

2.  $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$

3.  $(\neg B_{1,1} \vee (P_{1,2} \vee P_{2,1})) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$

4.  $(\neg B_{1,1} \vee (P_{1,2} \vee P_{2,1})) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$

5.  $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

6.  $\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\}$



# L'algoritmo DPLL per la soddisfacibilità

- **DPLL**: Davis, Putman, e poi Logemann, Loveland
- Parte da una KB **in forma a clausole**
  - prende in input una formula in CNF (cioè, un insieme di clausole)
- È una enumerazione ricorsiva *in profondità* di tutte le possibili interpretazioni alla ricerca di un modello
- Tre miglioramenti rispetto a TV-Consegue?:
  1. Terminazione anticipata
  2. Euristica dei simboli (o letterali) puri
  3. Euristica delle clausole unitarie

# DPLL: terminazione anticipata

- Si può decidere sulla verità di una clausola anche con interpretazioni parziali: basta che un letterale sia vero
  - Se  $A$  è vero, lo sono anche  $\{A, B\}$  e  $\{A, C\}$  indipendentemente dai valori di  $B$  e  $C$
- Se anche una sola clausola è falsa l'interpretazione non può essere un modello dell'insieme di clausole
  - Dato  $\{C_1\}\{C_2\}$ , se  $C_1$  è falso lo è anche  $\{C_1\}\{C_2\}$

(cioè  $C_1 \wedge C_2$ )

# DPLL: simboli puri

*Simbolo puro*: un simbolo che appare con lo stesso «segno» in tutte le clausole

Es.  $\{A, \neg B\} \{\neg B, \neg C\} \{C, A\}$  A è puro, B anche

- Nel determinare se un simbolo è puro se ne possono trascurare le occorrenze in clausole già rese vere
- I simboli puri possono essere assegnati a *True* se il letterale è positivo, *False* se negativo.
- Non si eliminano modelli utili: se le clausole hanno un modello continuano ad averlo dopo questo assegnamento (non renderà mai falsa una clausola in cui compare il simbolo puro)

# DPLL: clausole unitarie

*Clausola unitaria*: una clausola con un solo letterale *non assegnato*

Es.  $\{B\}$  è unitaria ma anche ...

$\{B, \neg C\}$  è unitaria quando  $C = \text{True}$

- Conviene assegnare prima valori al letterale in clausole unitarie. L'assegnamento è obbligato (*True* se positivo, *False* se negativo).

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return** *true*

**if** qualche clausola in *clausole* è falsa in *modello* **then return** *false*

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return**           DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

                  DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return** *true*

**if** qualche clausola in *clausole* è falsa in *modello* **then return** *false*

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return**           DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

                  DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return** *true*

**if** qualche clausola in *clausole* è falsa in *modello* **then return** *false*

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return** DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return** *true*

**if** qualche clausola in *clausole* è falsa in *modello* **then return** *false*

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return** DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return** *true*

**if** qualche clausola in *clausole* è falsa in *modello* **then return** *false*

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return**           DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

                  DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return** *true*

**if** qualche clausola in *clausole* è falsa in *modello* **then return** *false*

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return** DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

        DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

DPLL prende un insieme di clausole, un insieme di simboli da assegnare (inizialmente tutti i simboli presenti nelle clausole), e un assegnamento di verità ai simboli, inizialmente vuoto.

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return** *true*

**if** qualche clausola in *clausole* è falsa in *modello* **then return** *false*

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return** DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

        DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return true**

**if** qualche clausola in *clausole* è falsa in *modello* **then return false**

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return** DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

Si controlla che tutte le clausole siano soddisfatte dall'assegnamento, anche parziale, in tal caso si restituisce vero.

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return true**

**if qualche clausola in *clausole* è falsa in *modello* then return false**

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return** DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

Si controlla se (anche solo) una delle clausole non è soddisfatta, in tal caso falso.

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return** *true*

**if** qualche clausola in *clausole* è falsa in *modello* **then return** *false*

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return** DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

se esiste un simbolo puro, lo restituisce insieme al suo valore (True, se compare positivo, False, se compare negato)

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return true**

**if** qualche clausola in *clausole* è falsa in *modello* **then return false**

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return** DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

Se esiste viene richiamato DPLL dopo avere esteso il modello con l'assegnamento al simbolo puro e aver tolto il simbolo dai simboli ancora da assegnare.

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return true**

**if** qualche clausola in *clausole* è falsa in *modello* **then return false**

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return** DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

se esiste una clausola unitaria restituisce il simbolo della clausola unitaria e il suo valore (True, se compare positivo, False, se compare negato)

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return true**

**if** qualche clausola in *clausole* è falsa in *modello* **then return false**

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return** DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**

DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

Se esiste viene richiamato DPLL dopo avere esteso il modello con l'assegnamento al simbolo puro e aver tolto il simbolo dai simboli ancora da assegnare.

# L'algoritmo DPLL

**function** DPLL-Soddisfacibile?(*s*) **returns** *true* oppure *false*

**inputs:** *s*, una formula della logica proposizionale

*clausole*  $\leftarrow$  l'insieme di clausole nella rappresentazione CNF di *s*

*simboli*  $\leftarrow$  una lista di tutti i simboli proposizionali in *s*

**return** DPLL(*clausole*, *simboli*, { })

**function** DPLL(*clausole*, *simboli*, *modello*) **returns** *true* oppure *false*

**if** ogni clausola in *clausole* è vera in *modello* **then return true**

**if** qualche clausola in *clausole* è falsa in *modello* **then return false**

*P*, *valore*  $\leftarrow$  Trova-Simbolo-Puro(*simboli*, *clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*, *valore*  $\leftarrow$  Trova-Clausola-Unitaria(*clausole*, *modello*)

**if** *P* è diverso da null **then return** DPLL(*clausole*, *simboli* – *P*, *modello*  $\leftarrow$  {*P* = *valore*})

*P*  $\leftarrow$  Primo(*simboli*); *resto*  $\leftarrow$  Resto(*simboli*)

**return** DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *true*}) **or**  
DPLL(*clausole*, *resto*, *modello*  $\leftarrow$  {*P* = *false*})

Altrimenti si prende il primo simbolo da assegnare e ricorsivamente si richiama DPLL provando ad assegnare Vero o Falso. Se uno dei due ha successo si termina con successo.

# DPLL: esempio

Vogliamo verificare se

$$\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\} \{\neg B_{1,1}\} \models \{\neg P_{1,2}\}$$

KB

$\alpha$

# DPLL: esempio

Vogliamo verificare se

$$\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\} \{\neg B_{1,1}\} \models \{\neg P_{1,2}\}$$

Aggiungiamo  $\{P_{1,2}\}$  e vediamo se l'insieme è insoddisfacibile

$$\text{SAT}(\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\} \{\neg B_{1,1}\} \{P_{1,2}\}) ?$$

# DPLL: esempio

KB  $\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\} \{\neg B_{1,1}\} \models \{\neg P_{1,2}\} ?$

Aggiungiamo  $\{P_{1,2}\}$  e vediamo se l'insieme è insoddisfacibile

SAT(  $\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\} \{\neg B_{1,1}\} \{P_{1,2}\} ) ?$

1

2

3

4

5

1. La 5 è unitaria;  $P_{1,2} = \text{True}$ ; la prima clausola e la 5 sono soddisfatte

$\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\} \{\neg B_{1,1}\} \{P_{1,2}\}$

2.  $P_{2,1}$  è un simbolo puro;  $P_{2,1} = \text{False}$ ; 3 è soddisfatta

$\{\neg B_{1,1}, P_{1,2}, P_{2,1}\} \{\neg P_{1,2}, B_{1,1}\} \{\neg P_{2,1}, B_{1,1}\} \{\neg B_{1,1}\} \{P_{1,2}\}$

...

Non esistono modelli quindi  $\neg P_{1,2}$  è conseguenza logica della KB



# Miglioramenti di DPLL

- DPLL è **completo** e **termina** sempre
- Alcuni **miglioramenti**:
  - Analisi di componenti (sotto-problemi indipendenti): se le variabili possono essere suddivise in sotto-insiemi disgiunti (senza simboli in comune)
    - ✓ un risolutore può lavorare separatamente su ciascun componente
  - Ordinamento di variabili e valori: **scegliere** la variabile che **compare** in **più** clausole
  - Backtracking intelligente, riavvio casuale e altre ottimizzazioni ...

# Metodi locali per SAT: formulazione

- Gli 'stati' sono interpretazioni, assegnamenti completi
- L'obiettivo è un assegnamento che soddisfa tutte le clausole (un modello)
- Si parte da un assegnamento casuale
- Ad ogni passo si cambia il valore di un simbolo proposizionale (*flip*)
- Gli stati sono valutati contando il numero di clausole soddisfatte (più sono meglio è)

# Metodi locali per SAT: algoritmi

- Ci sono molti massimi locali per sfuggire ai quali serve introdurre perturbazioni casuali
  - Hill climbing con riavvio casuale
  - Simulated Annealing
- Molta sperimentazione per trovare il miglior compromesso tra il grado di “avidità” e casualità
- WALK-SAT è uno degli algoritmi più semplici ed efficaci



# WalkSAT

WalkSAT ad ogni passo

- Sceglie a caso una clausola non ancora soddisfatta
- Individua un simbolo da modificare (*flip*), scegliendo con probabilità  $p$  (di solito 0.5) tra:
  - ✓ Scegliere un simbolo a caso (*passo casuale, random walk*)
  - ✓ Scegliere quello che rende più clausole soddisfatte (*passo di ottimizzazione*)
- Si arrende dopo un certo numero di *flip* predefinito

# WalkSat: l'algoritmo

```
function WalkSAT(clausole, p, max_flips) returns un modello o fallimento
  inputs: clausole, un insieme di clausole della logica proposizionale
  p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
  max_flips, numero massimo di inversioni di valore prima di abbandonare

  modello  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

  for i = 1 to max_flips do
    if modello soddisfa clausole then return modello
    clausola  $\leftarrow$  una clausola, falsa in modello, scelta casualmente nell'insieme clausole
    if Random(0, 1)  $\leq$  p then inverti il valore in modello di un simbolo scelto
      casualmente in clausola
    else   inverti il valore di verità del simbolo in clausole che massimizza il numero
      di clausole soddisfatte

  return fallimento
```

# WalkSat: l'algoritmo

**function** WalkSAT(*clausole*, *p*, *max\_flips*) **returns** un modello o fallimento

**inputs:** *clausole*, un insieme di clausole della logica proposizionale  
*p*, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5  
*max\_flips*, numero massimo di inversioni di valore prima di abbandonare

*modello*  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in *clausole*

**for** *i* = 1 **to** *max\_flips* **do**

**if** *modello* soddisfa *clausole* **then return** *modello*

*clausola*  $\leftarrow$  una clausola, falsa in *modello*, scelta casualmente nell'insieme *clausole*

**if** Random(0, 1)  $\leq$  *p* **then** inverti il valore in *modello* di un simbolo scelto  
        casualmente in *clausola*

**else** inverti il valore di verità del simbolo in *clausole* che massimizza il numero  
        di clausole soddisfatte

**return** fallimento

# WalkSat: l'algoritmo

**function** WalkSAT(*clause*, *p*, *max flips*) **returns** un modello o fallimento

**inputs:** *clause*, un insieme di clausole della logica proposizionale  
 $p$ , la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5  
*max\_flips*, numero massimo di inversioni di valore prima di abbandonare

*modello*  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

```
for i = 1 to max_flips do
```

**if** *modello* soddisfa clausole **then return** *modello*

*clausola*  $\leftarrow$  una clausola, falsa in *modello*, scelta casualmente nell'insieme *clausole*

**if** Random(0, 1)  $\leq$  p **then** inverti il valore in *modello* di un simbolo scelto casualmente in *clausola*

**else**    inverti il valore di verità del simbolo in *clausole* che massimizza il numero di clausole soddisfatte

```
return fallimento
```

# WalkSat: l'algoritmo

**function** WalkSAT(*clausole*, *p*, *max\_flips*) **returns** un modello o fallimento  
**inputs:** *clausole*, un insieme di clausole della logica proposizionale  
*p*, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5  
*max\_flips*, numero massimo di inversioni di valore prima di abbandonare

*modello*  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in *clausole*

**for** *i* = 1 **to** *max\_flips* **do**  
    **if** *modello* soddisfa *clausole* **then return** *modello*  
    *clausola*  $\leftarrow$  una clausola, falsa in *modello*, scelta casualmente nell'insieme *clausole*  
    **if** Random(0, 1)  $\leq$  *p* **then** inverti il valore in *modello* di un simbolo scelto casualmente in *clausola*  
    **else** inverti il valore di verità del simbolo in *clausole* che massimizza il numero di clausole soddisfatte  
  
**return** fallimento

# WalkSat: l'algoritmo

**function** WalkSAT(*clausole*, *p*, *max\_flips*) **returns** un modello o fallimento  
**inputs:** *clausole*, un insieme di clausole della logica proposizionale  
*p*, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5  
*max\_flips*, numero massimo di inversioni di valore prima di abbandonare

*modello*  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in *clausole*

**for** *i* = 1 **to** *max\_flips* **do**

**if** *modello* soddisfa *clausole* **then return** *modello*

*clausola*  $\leftarrow$  una clausola, falsa in *modello*, scelta casualmente nell'insieme *clausole*

**if** Random(0, 1)  $\leq$  *p* **then** inverti il valore in *modello* di un simbolo scelto  
        casualmente in *clausola*

**else** inverti il valore di verità del simbolo in *clausole* che massimizza il numero  
        di clausole soddisfatte

**return** fallimento

# WalkSat: l'algoritmo

```
function WalkSAT(clausole, p, max_flips) returns un modello o fallimento
  inputs: clausole, un insieme di clausole della logica proposizionale
  p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
  max_flips, numero massimo di inversioni di valore prima di abbandonare

  modello  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

  for i = 1 to max_flips do
    if modello soddisfa clausole then return modello
    clausola  $\leftarrow$  una clausola, falsa in modello, scelta casualmente nell'insieme clausole
    if Random(0, 1)  $\leq$  p then inverti il valore in modello di un simbolo scelto
      casualmente in clausola
    else inverti il valore di verità del simbolo in clausole che massimizza il numero
      di clausole soddisfatte

  return fallimento
```

# WalkSat: l'algoritmo

```
function WalkSAT(clausole, p, max_flips) returns un modello o fallimento
  inputs: clausole, un insieme di clausole della logica proposizionale
  p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
  max_flips, numero massimo di inversioni di valore prima di abbandonare

  modello  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

  for i = 1 to max_flips do
    if modello soddisfa clausole then return modello
    clausola  $\leftarrow$  una clausola, falsa in modello, scelta casualmente nell'insieme clausole
    if Random(0, 1)  $\leq$  p then inverti il valore in modello di un simbolo scelto
      casualmente in clausola
    else inverti il valore di verità del simbolo in clausole che massimizza il numero
      di clausole soddisfatte

  return fallimento
```

# WalkSat: l'algoritmo

**function** WalkSAT(*clausole*, *p*, *max\_flips*) **returns** un modello o fallimento

**inputs:** *clausole*, un insieme di clausole della logica proposizionale

*p*, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5

*max\_flips*, numero massimo di inversioni di valore prima di abbandonare

*modello*  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in *clausole*

**for** *i* = 1 **to** *max\_flips* **do**

**if** *modello* soddisfa *clausole* **then return** *modello*

*clausola*  $\leftarrow$  una clausola falsa in *modello*, scelta casualmente nell'insieme *clausole*

**if** Random(0, 1)  $\leq$  *p* **then** inverti il valore in *modello* di un simbolo scelto casualmente in *clausola*

**else** inverti il valore di verità del simbolo in *clausole* che massimizza il numero di clausole soddisfatte

**return** fallimento

con probabilità *p*: random walk

# WalkSat: l'algoritmo

```
function WalkSAT(clausole, p, max_flips) returns un modello o fallimento
  inputs: clausole, un insieme di clausole della logica proposizionale
  p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
  max_flips, numero massimo di inversioni di valore prima di abbandonare

  modello  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

  for i = 1 to max_flips do
    if modello soddisfa clausole then return modello
    clausola  $\leftarrow$  una clausola, falsa in modello, scelta casualmente nell'insieme clausole
    if Random(0, 1)  $\leq$  p then inverti il valore in modello di un simbolo scelto
      casualmente in clausola
    else inverti il valore di verità del simbolo in clausole che massimizza il numero
      di clausole soddisfatte

  return fallimento
```

con probabilità (1-p): ottimizzazione

# WalkSat: l'algoritmo

```
function WalkSAT(clausole, p, max_flips) returns un modello o fallimento
  inputs: clausole, un insieme di clausole della logica proposizionale
  p, la probabilità di effettuare una “camminata casuale”, tipicamente intorno a 0,5
  max_flips, numero massimo di inversioni di valore prima di abbandonare

  modello  $\leftarrow$  un assegnamento casuale di valori di verità ai simboli in clausole

  for i = 1 to max_flips do
    if modello soddisfa clausole then return modello
    clausola  $\leftarrow$  una clausola, falsa in modello, scelta casualmente nell'insieme clausole
    if Random(0, 1)  $\leq$  p then inverti il valore in modello di un simbolo scelto
      casualmente in clausola
    else   inverti il valore di verità del simbolo in clausole che massimizza il numero
      di clausole soddisfatte

  return fallimento
```

# WalkSAT: un esempio

Come euristica usiamo il numero di clausole soddisfatte (da massimizzare)

Rosso: passo casuale

Verde: passo di ottimizzazione

$$\begin{array}{cccc} \{\neg B_{1,1}, P_{1,2}, P_{2,1}\} & \{\neg P_{1,2}, B_{1,1}\} & \{\neg P_{2,1}, B_{1,1}\} & \{\neg B_{1,1}\} \\ 1 & 2 & 3 & 4 \end{array}$$

[ $B_{1,1}=F, P_{1,2}=T, P_{2,1}=T$ ] 2, 3 F; scelgo 2; a caso flip  $B_{1,1}$

[ $B_{1,1}=T, P_{1,2}=T, P_{2,1}=T$ ] 4 F; scelgo 4; flip  $B_{1,1}$  unica scelta

[ $B_{1,1}=F, P_{1,2}=T, P_{2,1}=T$ ] 2, 3 F; scelgo 2; a caso flip  $P_{1,2}$

[ $B_{1,1}=F, P_{1,2}=F, P_{2,1}=T$ ] 3 F; scelgo 3; ottimizzazione: flip  $P_{2,1}$ [#4]; flip  $B_{1,1}$ [#3]

[ $B_{1,1}=F, P_{1,2}=F, P_{2,1}=F$ ] ho trovato un modello!!!

# Analisi di WalkSAT

- Se  $max-flips = \infty$  e l'insieme di clausole è soddisfacibile prima o poi termina.
- Ma se vogliamo un algoritmo che termina, questo è necessariamente incompleto
- Va bene per cercare un modello, sapendo che c'è, ma se è insoddisfacibile non termina
- Non può essere usato per verificare l'insoddisfacibilità

# Problemi SAT difficili

*cioè con un numero relativamente  
piccolo di clausole*

- Se un problema ha molte soluzioni (problema sotto-vincolato) è più probabile che WalkSAT ne trovi una in tempi brevi
- Istanza di 3-SAT (clausole con tre letterali)  
 $\{\neg D, \neg B, C\} \{B, \neg A, \neg C\} \{\neg C, \neg B, E\} \{E, \neg D, B\} \{B, E, \neg C\}$ 
  - Esempio: 16 soluzioni su 32; un assegnamento ha il 50% di probabilità di essere giusto: 2 passi in media!

# Problemi SAT difficili

- Se un problema ha molte soluzioni (problema sotto-vincolato) è più probabile che WalkSAT ne trovi una in tempi brevi
- Istanza di 3-SAT (clausole con tre letterali)

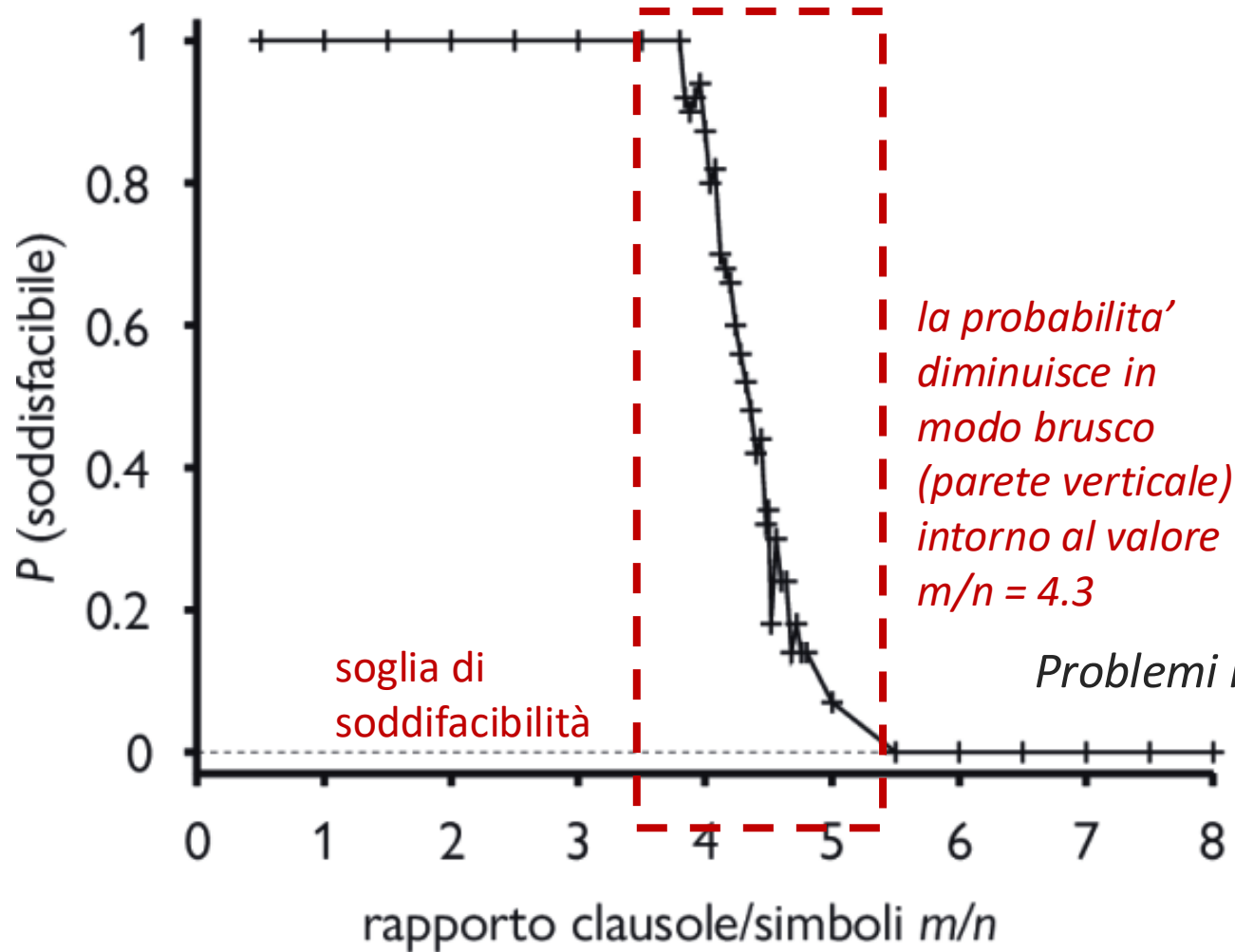
$\{\neg D, \neg B, C\} \{B, \neg A, \neg C\} \{\neg C, \neg B, E\} \{E, \neg D, B\} \{B, E, \neg C\}$

- Esempio: 16 soluzioni su 32; un assegnamento ha il 50% di probabilità di essere giusto: 2 passi in media!

- È significativo il rapporto  $m/n$ 
  - $m$  è il numero di clausole (vincoli)
  - $n$  il numero di simboli
  - Nell'esempio:  $5/5=1$
- Più grande il rapporto, più vincolato è il problema

# Probabilità di soddisfacibilità in funzione di $m/n$

*Problemi poco vincolati*



$m$  (n. clausole) varia

$n$  (n. simboli) = 50

3 letterali per clausola

media su 100

problemi generati a caso

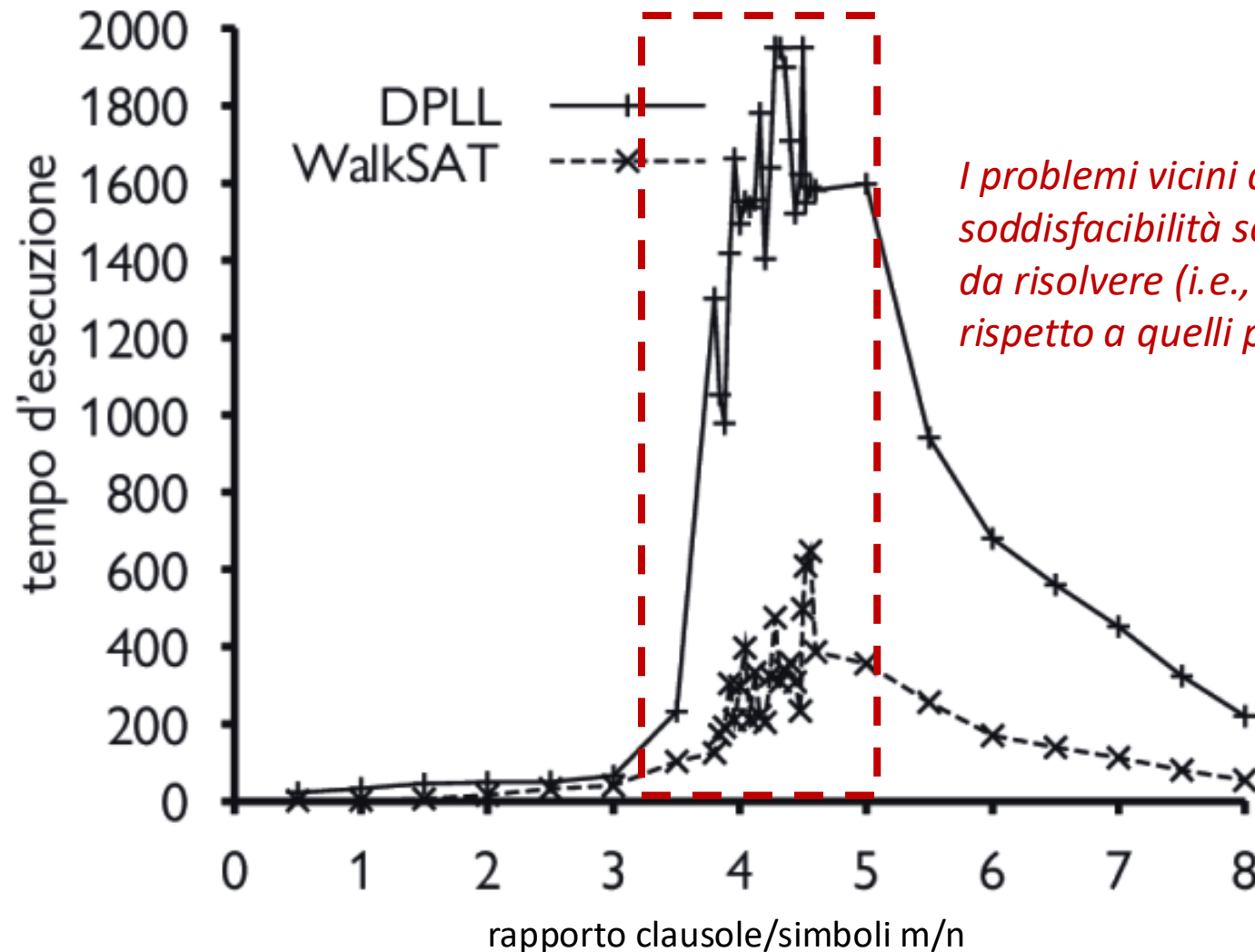
*la probabilità  
diminuisce in  
modo brusco  
(parete verticale)  
intorno al valore  
 $m/n = 4.3$*

*Problemi molto vincolati*

Confronto su problemi soddisfacibili, ma difficili

# Confronto tra DPLL e WalkSAT

tempo medio di  
esecuzione (misurato in  
numero di iterazioni)  
per DPLL e WalkSAT su  
formule casuali 3-CNF



*I problemi vicini alla soglia di  
soddisfacibilità sono molto più difficili  
da risolvere (i.e., richiedono più tempo)  
rispetto a quelli più lontani*